CM03 - C Introduction, Structure, Tools

C motivation, multi-file programs, compilation, make, debugging

Louis Ledoux

2026-07-05

Table of contents

1 Why C

Live pacing: about 8 min

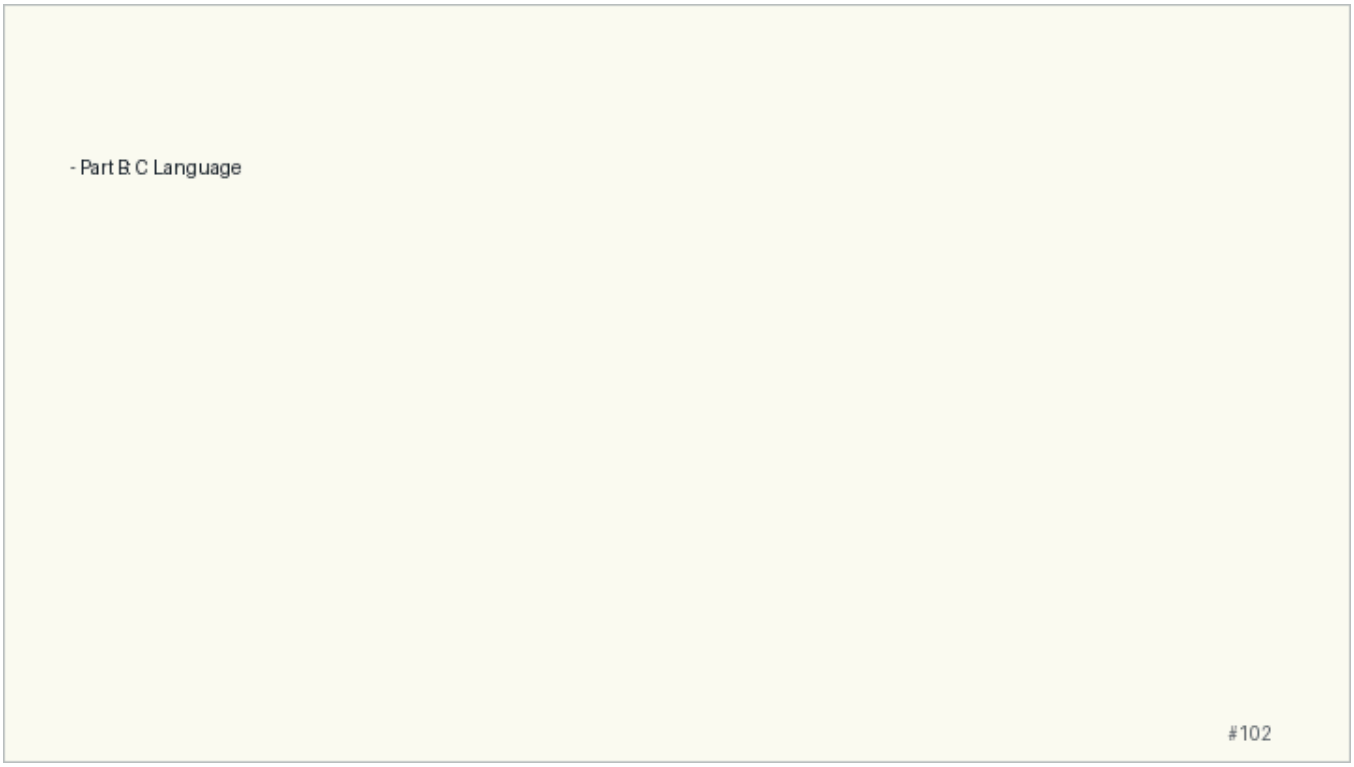
Question. Why learn C when higher-level languages exist?

Core claim. C exposes representation, memory, compilation, and linking decisions that other languages hide.

Target capability. Students can distinguish declarations, definitions, source files, headers, and object files.

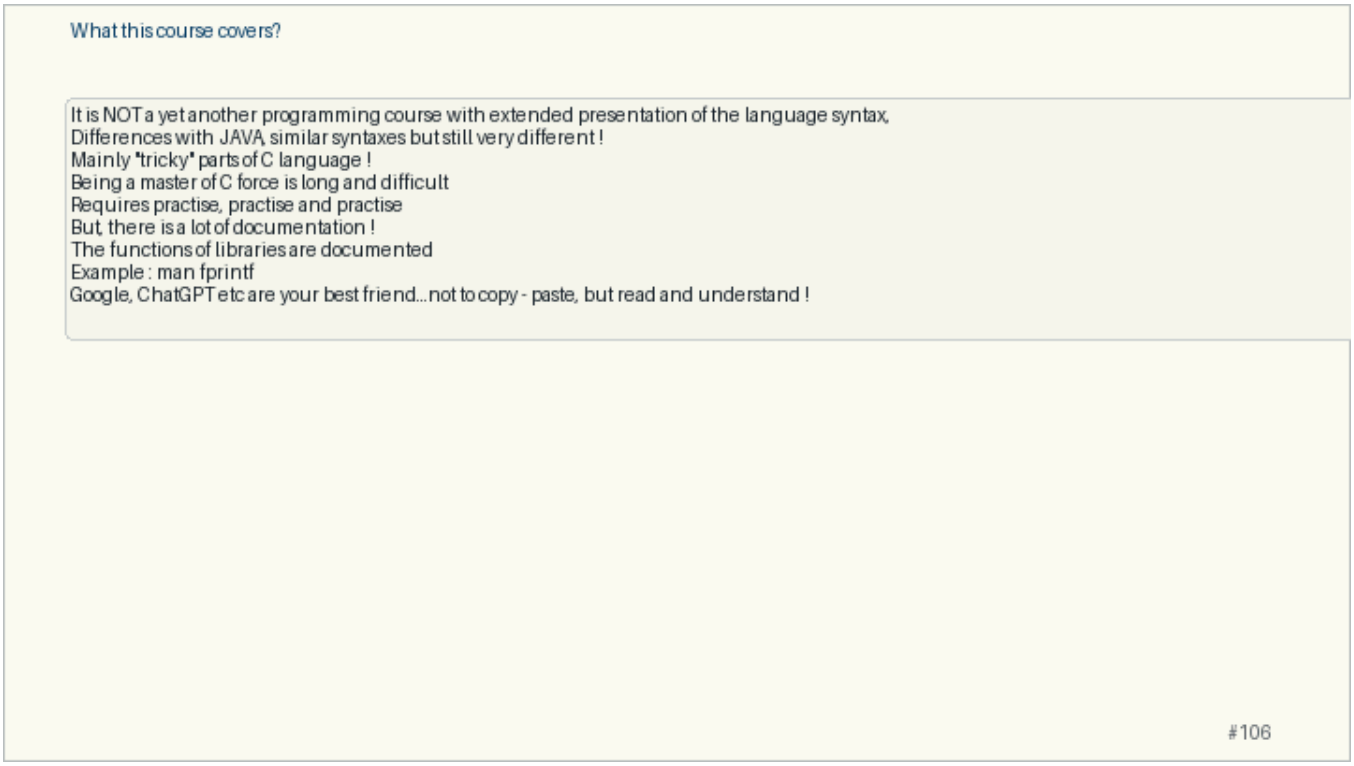
1.1 Main Path

- Start from the question: Why learn C when higher-level languages exist?
- Build the model: C exposes representation, memory, compilation, and linking decisions that other languages hide.
- End with a checkable action: Students can distinguish declarations, definitions, source files, headers, and object files.
- Keep only this slide on the horizontal path during the live lecture.



1.2 Worked Example

- Use this as the live trace: Split a tiny counter program into `.h`, `.c`, and `main.c`, then break one declaration deliberately.
- Representative source slide: 106
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.



1.3 Anecdote Or Motivation

C is close enough to the machine that small textual changes can change storage, calling, or linkage.

- Use this only if students need a reason to care.

- If the room is already following, skip down to the check question.

1.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

C language

- Developed by Thompson & Ritchie in 1972
- Originally developed: 90-95% of the Unix kernel
- It has been standardized by the American National Standards Institute (ANSI) in 1989 (C ANSI)
- Loosely/weakly typed language (so, warnings are important !)

#107

1.5 Backup Index

Original slides 102-107

- Part B: C Language

#102

slide 102

Outline

- Introduction
- Structure of C program and compilation
- Basic components of C language
- Pointers and Arrays
- Functions and parameters
- Dynamic memory allocation
- Structures, Lists and Hash tables
- Strings
- I/O

slide 103

- CM 3 Introduction, Structure and Tools

slide 104

Why C ?

JAVA is an evolved language
High abstraction away from the physical objects
Useful when we want to be abstracted from the hardware
Software engineering
C is closer the hardware
Relatively controls what happens to the memory
Manipulation of the physical objects of the hardware
System engineering
Main operating systems are written in C
Unix, GNU/Linux, etc.

slide 105

What this course covers?

It is NOT a yet another programming course with extended presentation of the language syntax,
Differences with JAVA, similar syntaxes but still very different !
Mainly "tricky" parts of C language !
Being a master of C force is long and difficult
Requires practise, practise and practise
But, there is a lot of documentation !
The functions of libraries are documented
Example : man fprintf
Google, ChatGPT etc are your best friend... not to copy - paste, but read and understand !

slide 106

C language

- Developed by Thompson & Ritchie in 1972
- Originally developed: 90-95% of the Unix kernel
- It has been standardized by the American National Standards Institute (ANSI) in 1989 (C ANSI)
- Loosely/weakly typed language (so, warnings are important !)

#107

slide 107

Anchors: Part B: C Language; Outline; CM 3 Introduction, Structure and Tools; Why C ?; What this course covers?; C language

1.6 Backup: Part B: C Language

Original slide 102

- Part B: C Language

#102

1.7 Backup: Outline

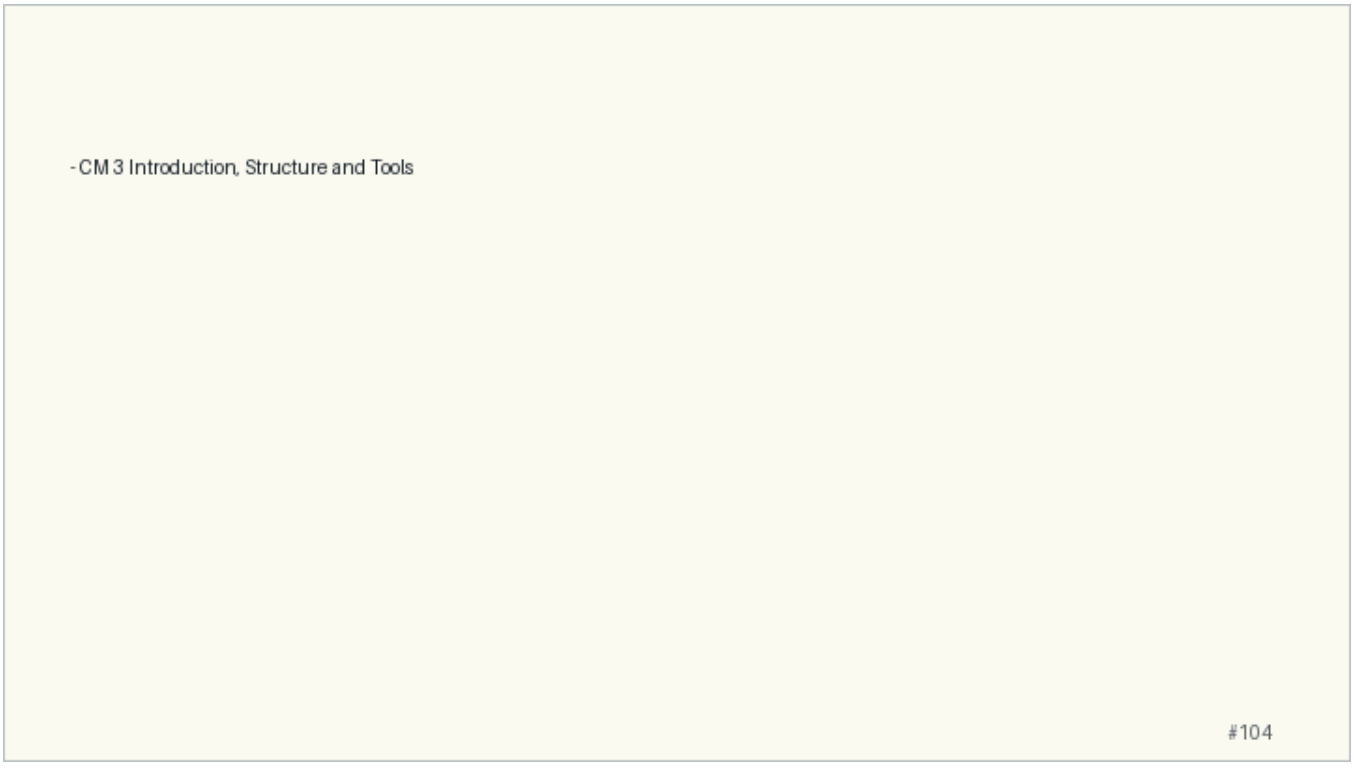
Original slide 103

- Introduction
- Structure of C program and compilation

- Basic components of C language
- Pointers and Arrays
- Functions and parameters
- Dynamic memory allocation
- Structures, Lists and Hash tables
- Strings
- I/O

1.8 Backup: CM 3 Introduction, Structure and Tools

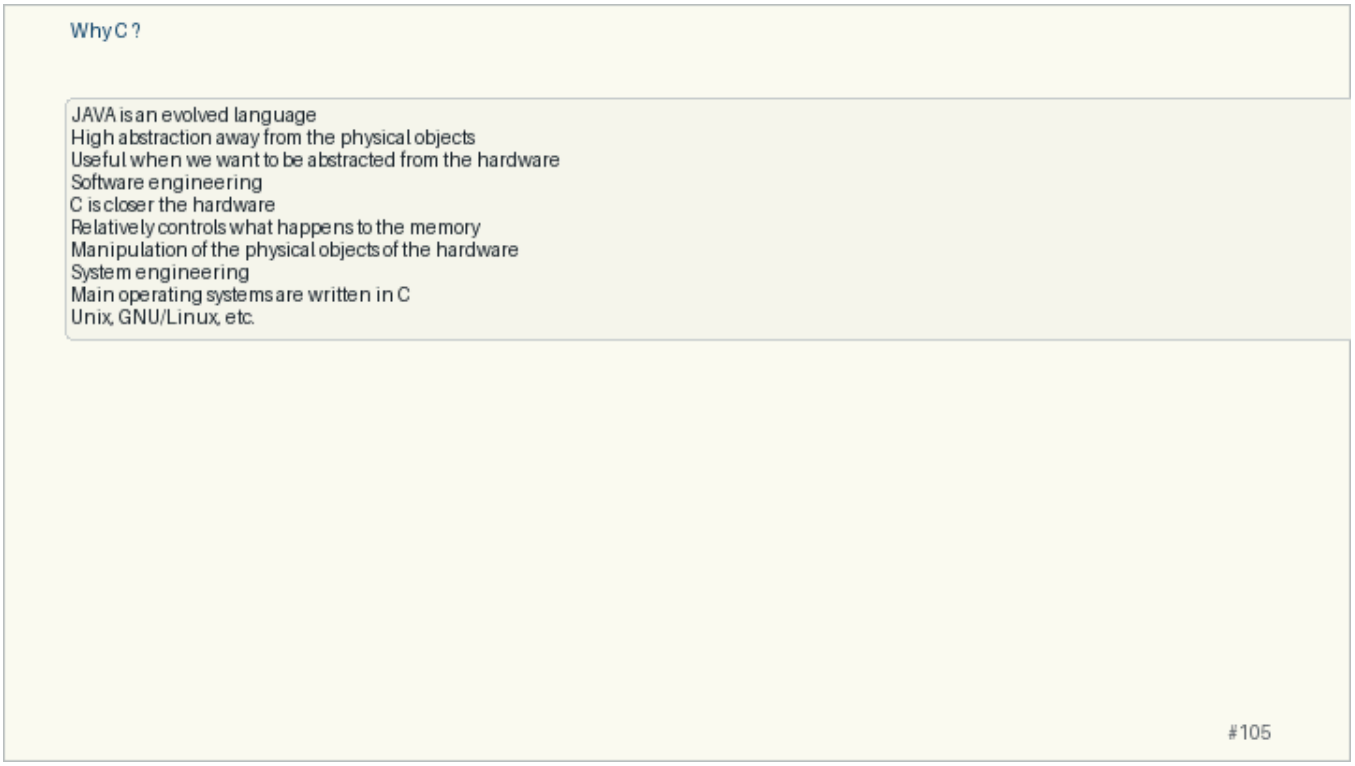
Original slide 104



1.9 Backup: Why C ?

Original slide 105

```
JAVA is an evolved language
High abstraction away from the physical objects
Useful when we want to be abstracted from the hardware
Software engineering
C is closer the hardware
Relatively controls what happens to the memory
Manipulation of the physical objects of the hardware
System engineering
Main operating systems are written in C
Unix, GNU/Linux, etc.
```



1.10 Backup: What this course covers?

Original slide 106

```
It is NOT a yet another programming course with extended presentation of the language syntax,
Differences with JAVA, similar syntaxes but still very different !
Mainly "tricky" parts of C language !
Being a master of C force is long and difficult
Requires practise, practise and practise
But, there is a lot of documentation !
The functions of libraries are documented
Example : man fprintf
Google, ChatGPT etc are your best friend...not to copy - paste, but read and understand !
```

What this course covers?

It is NOT a yet another programming course with extended presentation of the language syntax,
Differences with JAVA, similar syntaxes but still very different !
Mainly "tricky" parts of C language !
Being a master of C force is long and difficult
Requires practise, practise and practise
But, there is a lot of documentation !
The functions of libraries are documented
Example : man fprintf
Google, ChatGPT etc are your best friend...not to copy - paste, but read and understand !

#106

1.11 Backup: C language

Original slide 107

- Developed by Thompson & Ritchie in 1972
- Originally developed: 90-95% of the Unix kernel
- It has been standardized by the American National Standards Institute (ANSI) in 1989 (C ANSI)
- Loosely/weakly typed language (so, warnings are important !)

C language

- Developed by Thompson & Ritchie in 1972
- Originally developed: 90-95% of the Unix kernel
- It has been standardized by the American National Standards Institute (ANSI) in 1989 (C ANSI)
- Loosely/weakly typed language (so, warnings are important !)

#107

2 Program Structure

Live pacing: about 9 min

Question. Why learn C when higher-level languages exist?

Core claim. C exposes representation, memory, compilation, and linking decisions that other languages hide.

Target capability. Students can distinguish declarations, definitions, source files, headers, and object files.

2.1 Main Path

- Start from the question: Why learn C when higher-level languages exist?
- Build the model: C exposes representation, memory, compilation, and linking decisions that other languages hide.
- End with a checkable action: Students can distinguish declarations, definitions, source files, headers, and object files.
- Keep only this slide on the horizontal path during the live lecture.

2.2 Worked Example

- Use this as the live trace: Split a tiny counter program into `.h`, `.c`, and `main.c`, then break one declaration deliberately.
- Representative source slide: 113
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Communication among .c files: Example

```
#include "code1.h" /* Declaration made available here */
/* Variable defined and initialized here */
int global_variable = 1; /* Definition checked against declaration */
int increment(void) { return global_variable++; }
```

- code1.c

```
- extern int global_variable; /* Declaration of the variable */
```

- code1.h

```
#include "code1.h"
#include <stdio.h>
void use_it(void)
{
printf("Global variable: %d\n", global_variable);
}
```

- code2.c

#113

2.3 Anecdote Or Motivation

C is close enough to the machine that small textual changes can change storage, calling, or linkage.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

2.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Same example split in 3 files

```c
/*****************************
 * main.c
 *****************************/

#include <stdlib.h>
#include <stdio.h>

#include "ppcm.h"

#define PI 3.14

int variableGlobale = 3;

//extern int ppcm (int a, int b);

void printINT(int val)
{
    printf("%d\n",val);
}

int main(int argc, char *argv[])
{
    int variableLocale = ppcm(2,variableGlobale);
    printINT(variableLocale);
    return EXIT_SUCCESS;
}
```

```c
/*****************************
 * ppcm.h
 *****************************/

#ifndef PPCM_H
#define PPCM_H

int ppcm(int x, int y);

#endif
```

```c
/*****************************
 * ppcm.c
 *****************************/
int ppcm(int x, int y)
{
    int a=x;
    int b=y;
    while (a!=b)
    {
        while (a>b) b=b+y;
        while (a<b) a=a+x;
    }
    return a;
}
```

#120

2.5 Backup Index

Original slides 108-120

slide 108

slide 109

Communication among .c files

- code2.c can use the functions and the global variables defined:
- locally in this compilation file code2.c
- in another compilation file code1.c (code1.c exports them)
- code1.c defines the functions and the global variables:
- exported by code1.c to be used by another file code2.c
- used locally in code1.c

- code2.c

- Definition of variable b;
- Use of variable b;
- Use of variable a;

- code1.c

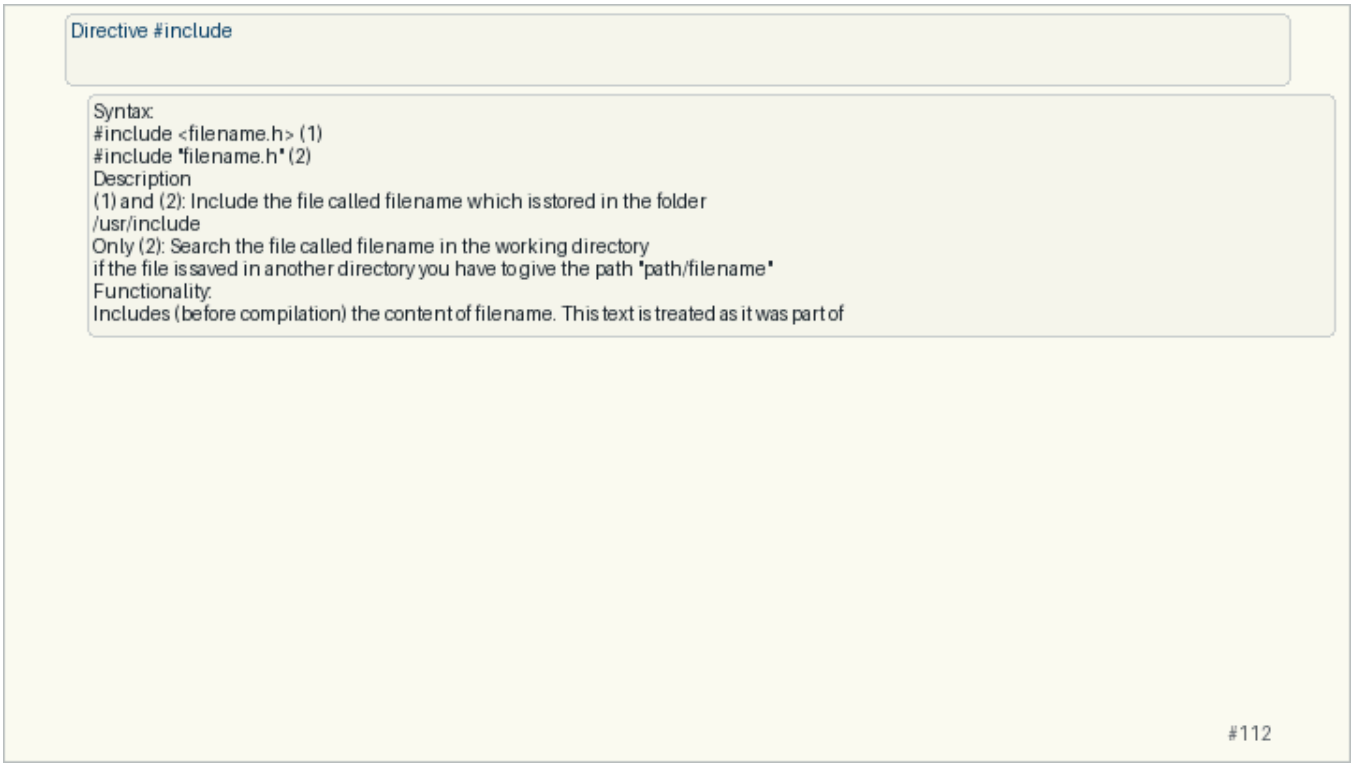
- Definition of variable a;
- Use of variable a;

slide 110

Communication among .c files

The definition of the global variables and constants is by default local:
defined in code1.c -> "seen" only by code1.c
To be "seen" by file code2.c -> the global variable has to be exported:
declarations are placed in the header file code1.h
extern: For the global variable to be seen by other .c files
definitions are in the source file code1.c
code2.c includes the header code1.h of code1.c through the directive #include

slide 111



slide 112

slide 113

Anchors: Structure of C program and compilation; C program's structure; Communication among .c files; Directive #include; Communication among .c files: Example; General rules

2.6 Backup: Structure of C program and compilation

Original slide 108

2.7 Backup: C program's structure

Original slide 109

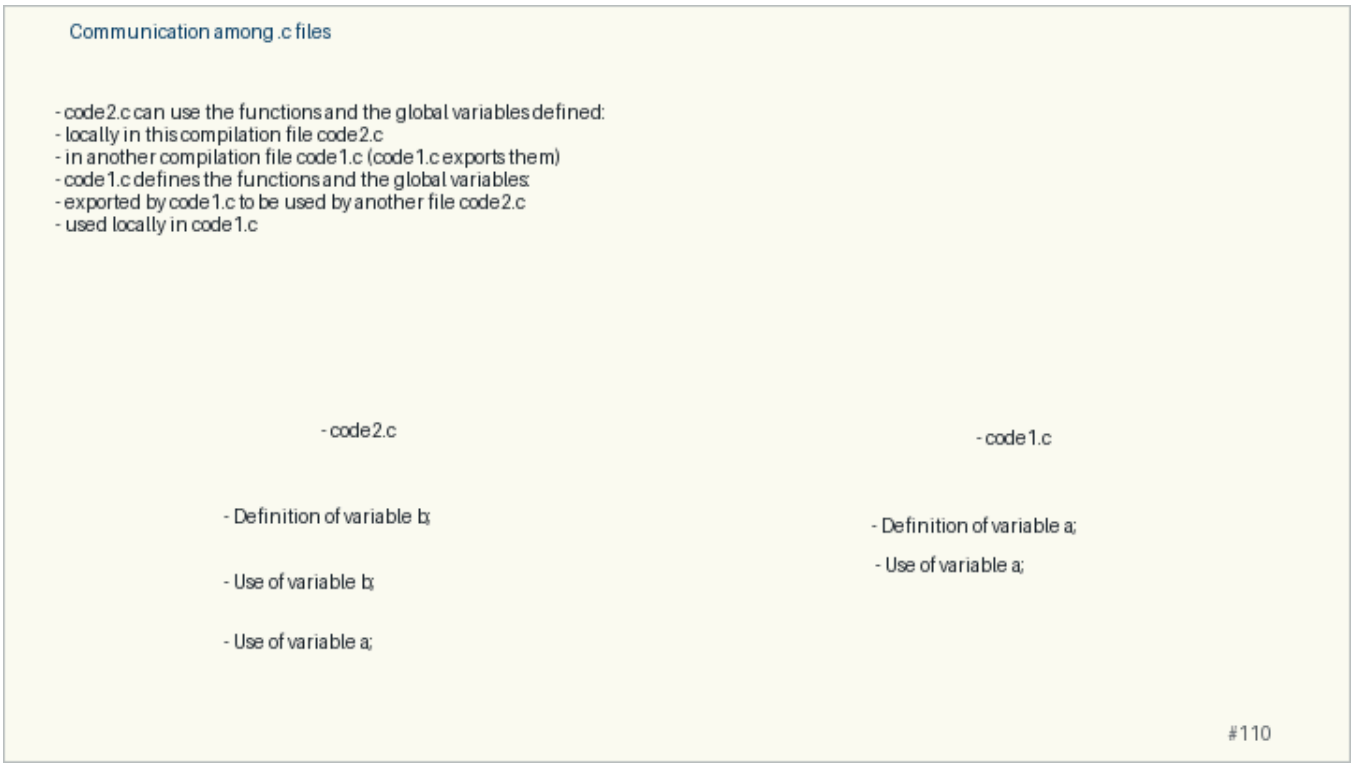
- A program is saved into one or multiple compilation C files
- One compilation file consist of a source file or a header file
- Source file .c -> code
- Header file .h -> declarations

2.8 Backup: Communication among .c files

Original slide 110

- code2.c can use the functions and the global variables defined:

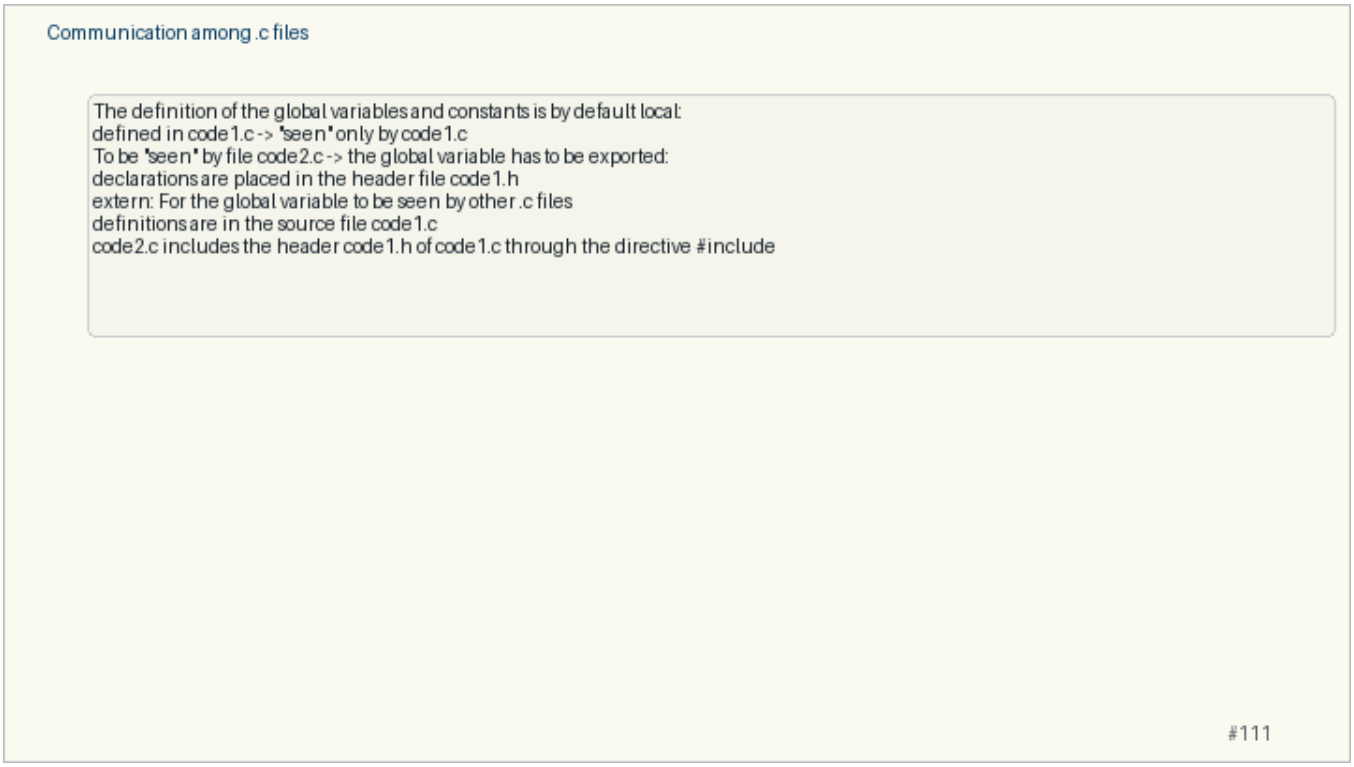
- locally in this compilation file code2.c
- in another compilation file code1.c (code1.c exports them)
- code1.c defines the functions and the global variables:
- exported by code1.c to be used by another file code2.c
- used locally in code1.c
- code2.c
- code1.c
- Definition of variable b;
- Definition of variable a;
- Use of variable a;
- Use of variable b;
- Use of variable a;



2.9 Backup: Communication among .c files

Original slide 111

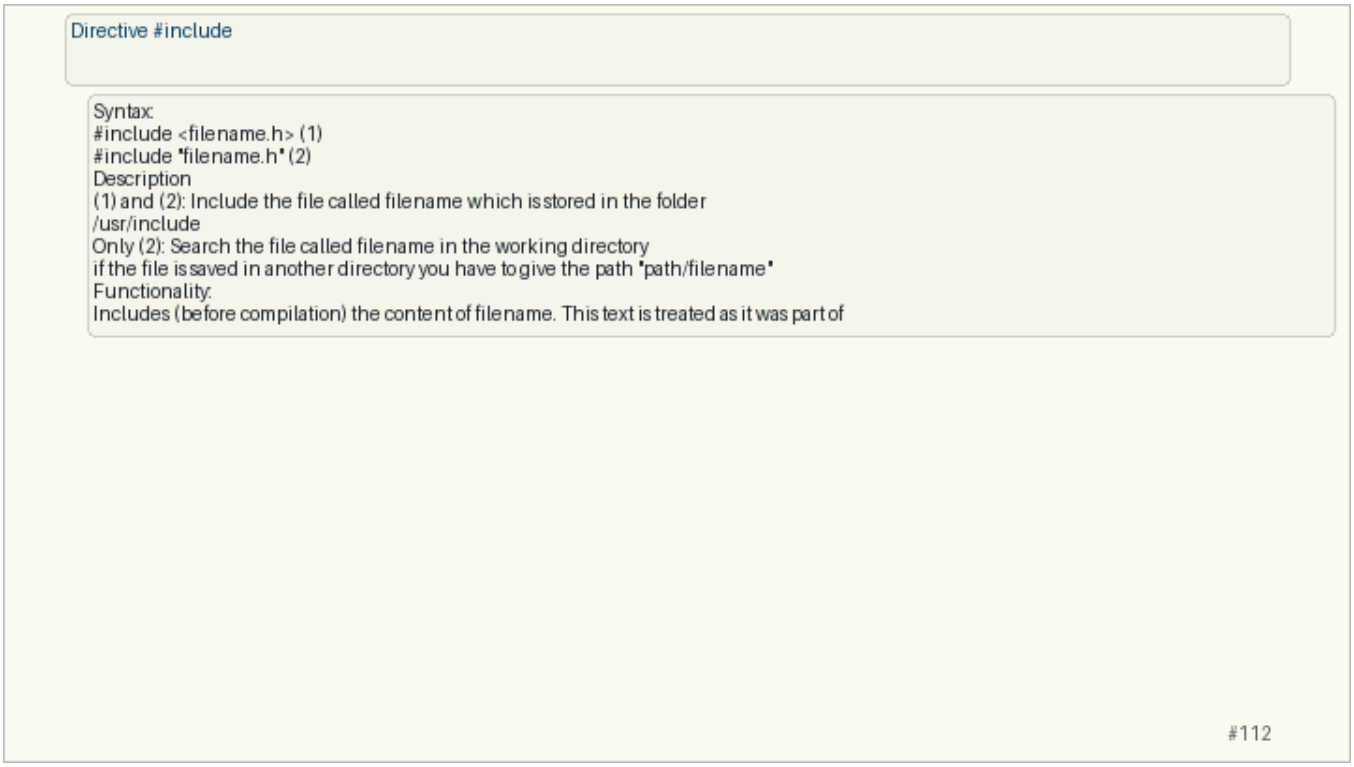
```
The definition of the global variables and constants is by default local:
defined in code1.c -> "seen" only by code1.c
To be "seen" by file code2.c -> the global variable has to be exported:
declarations are placed in the header file code1.h
extern: For the global variable to be seen by other .c files
definitions are in the source file code1.c
code2.c includes the header code1.h of code1.c through the directive #include
```



2.10 Backup: Directive #include

Original slide 112

```
Syntax:
#include <filename.h> (1)
#include "filename.h" (2)
Description
(1) and (2): Include the file called filename which is stored in the folder
/usr/include
Only (2): Search the file called filename in the working directory
if the file is saved in another directory you have to give the path "path/filename"
Functionality:
Includes (before compilation) the content of filename. This text is treated as it was part of
the current file .c
```



2.11 Backup: Communication among .c files: Example

Original slide 113

```c
#include "code1.h" /* Declaration made available here */
/* Variable defined and initialized here */
int global_variable = 1; /* Definition checked against declaration */
int increment(void) { return global_variable++; }
```

- code1.c
- extern int global_variable; /* Declaration of the variable */
- code1.h

```c
#include "code1.h"
#include <stdio.h>
void use_it(void)
{
printf("Global variable: %d\n", global_variable);
}
```

- code2.c

2.12 Backup: Communication among .c files: Example

Original slide 114

- extern void use_it(void);
- extern int increment(void);
- prog.h

```c
#include "code1.h"
#include "prog.h"
#include <stdio.h>
int main(void)
{
use_it();
global_variable += 10;
use_it();
increment();
use_it();
return 0;
}
```

- prog.c

2.13 Backup: Communication among .c files: Example

Original slide 115

```c
#include "code1.h" /* Declaration made available here */
#include "prog.h"
/* Variable defined here */
int global_variable = 1; /* Definition checked against declaration */
int increment(void) { return global_variable++; }
```

- code1.c
- extern int global_variable; /* Declaration of the variable */
- code1.h

```c
#include "code1.h"
#include "prog.h"
#include <stdio.h>
void use_it(void)
{
printf("Global variable: %d\n", global_variable++);
}
```

- code2.c

2.14 Backup: General rules

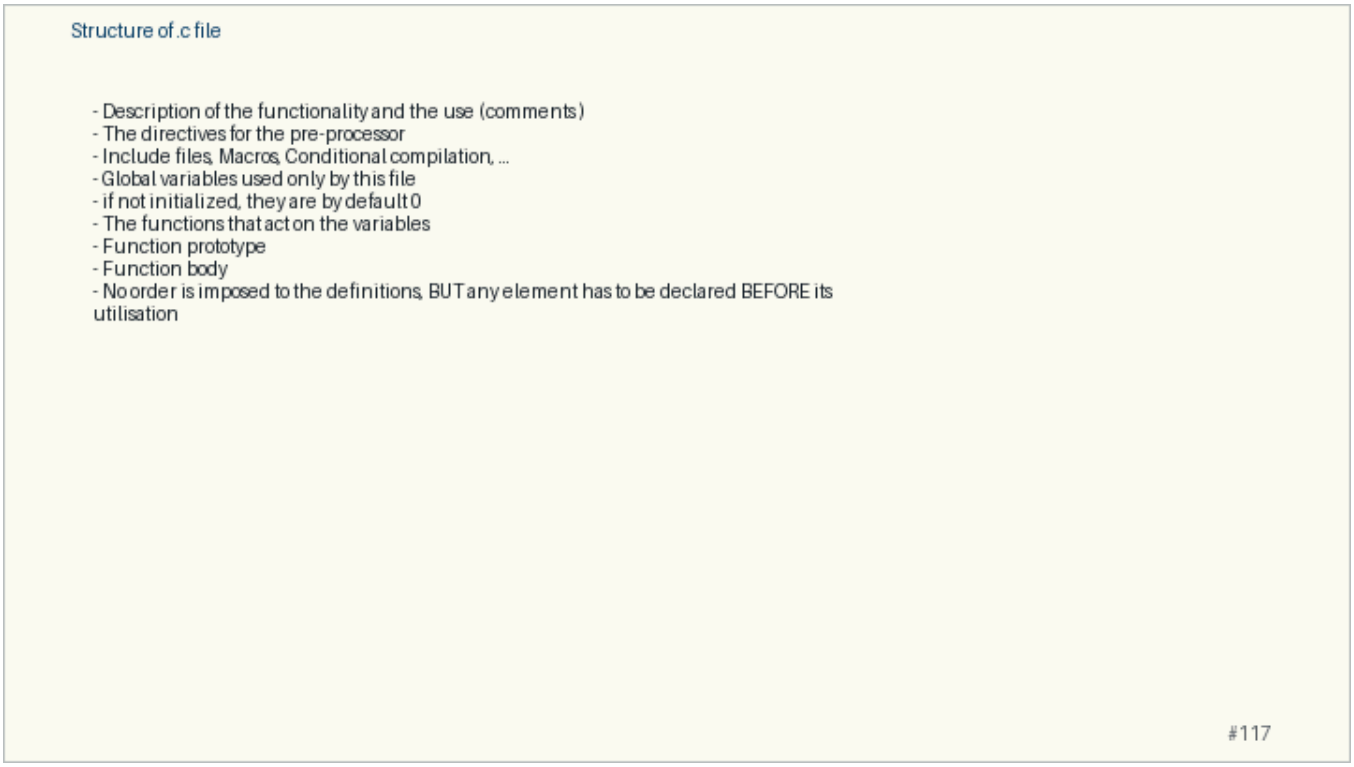
Original slide 116

- For any given global variable
- exactly 1 header file declares it
- exactly 1 source file defines it and preferably initializes it too.
- (otherwise undefined behavior -> anything could happen !)
- The source file that defines the variable also includes the header (ensuring definition and declaration are consistent)
- A source file
- never contains extern declarations of variables
- always include the (sole) header that declares them
- A header file only contains extern declarations of variables
- not static
- (each source file makes its own version of the global variable)

2.15 Backup: Structure of .c file

Original slide 117

- Description of the functionality and the use (comments)
- The directives for the pre-processor
- Include files, Macros, Conditional compilation, …
- Global variables used only by this file
- if not initialized, they are by default 0
- The functions that act on the variables
- Function prototype
- Function body
- No order is imposed to the definitions, BUT any element has to be declared BEFORE its utilisation



2.16 Backup: Structure of .h file

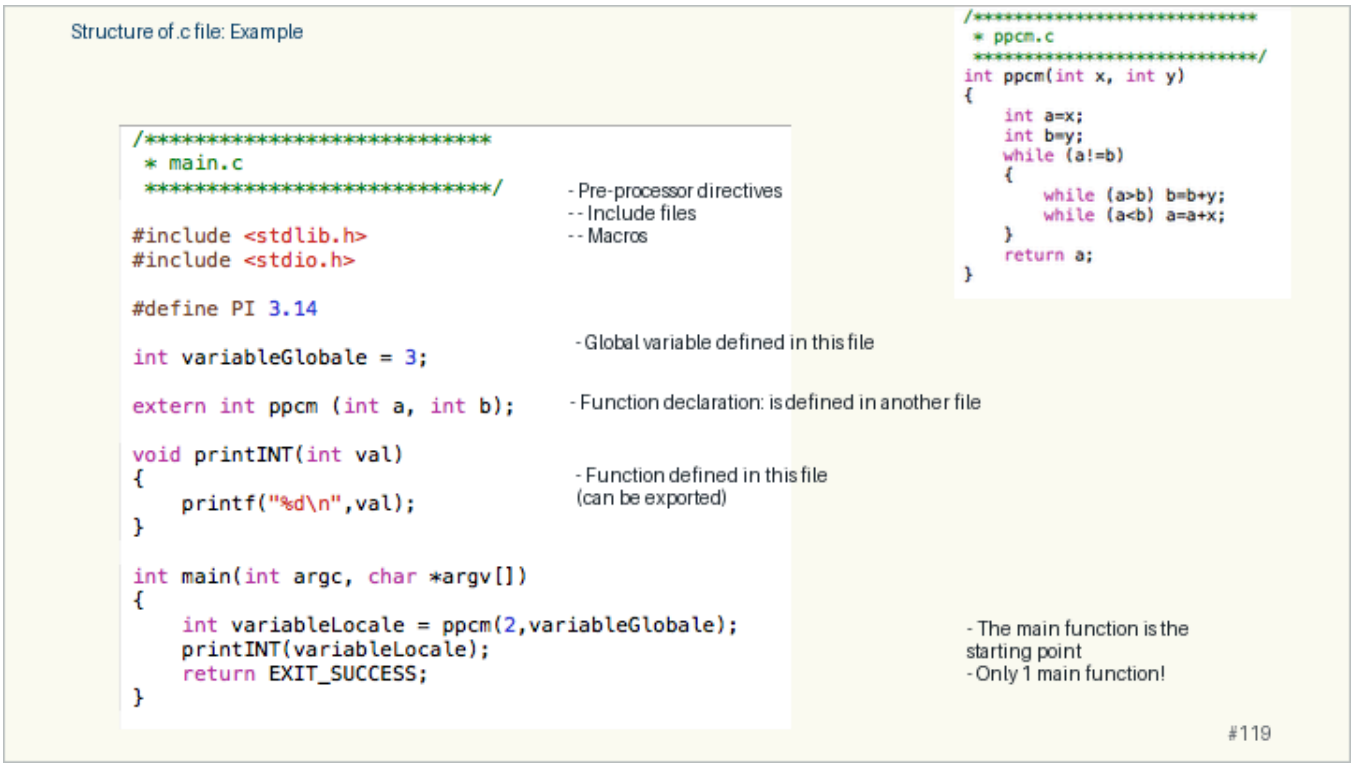
Original slide 118

- Declaration of whatever is allowed to be "seen" outside file .c
- Functions
- Global variables
- Data structures
- ...
- It is the source file .c interface to outside world

2.17 Backup: Structure of .c file: Example

Original slide 119

- Pre-processor directives
- ▸ Include files
- ▸ Macros
- Global variable defined in this file
- Function declaration: is defined in another file
- Function defined in this file (can be exported)
- The main function is the starting point
- Only 1 main function!



2.18 Backup: Same example split in 3 files

Original slide 120

Same example split in 3 files

```c
/*****************************
 * main.c
 *****************************/

#include <stdlib.h>
#include <stdio.h>

#include "ppcm.h"

#define PI 3.14

int variableGlobale = 3;

//extern int ppcm (int a, int b);

void printINT(int val)
{
    printf("%d\n",val);
}

int main(int argc, char *argv[])
{
    int variableLocale = ppcm(2,variableGlobale);
    printINT(variableLocale);
    return EXIT_SUCCESS;
}
```

```c
/*****************************
 * ppcm.h
 *****************************/

#ifndef PPCM_H
#define PPCM_H

int ppcm(int x, int y);

#endif
```

```c
/*****************************
 * ppcm.c
 *****************************/
int ppcm(int x, int y)
{
    int a=x;
    int b=y;
    while (a!=b)
    {
        while (a>b) b=b+y;
        while (a<b) a=a+x;
    }
    return a;
}
```

#120

3 Compilation

Live pacing: about 8 min

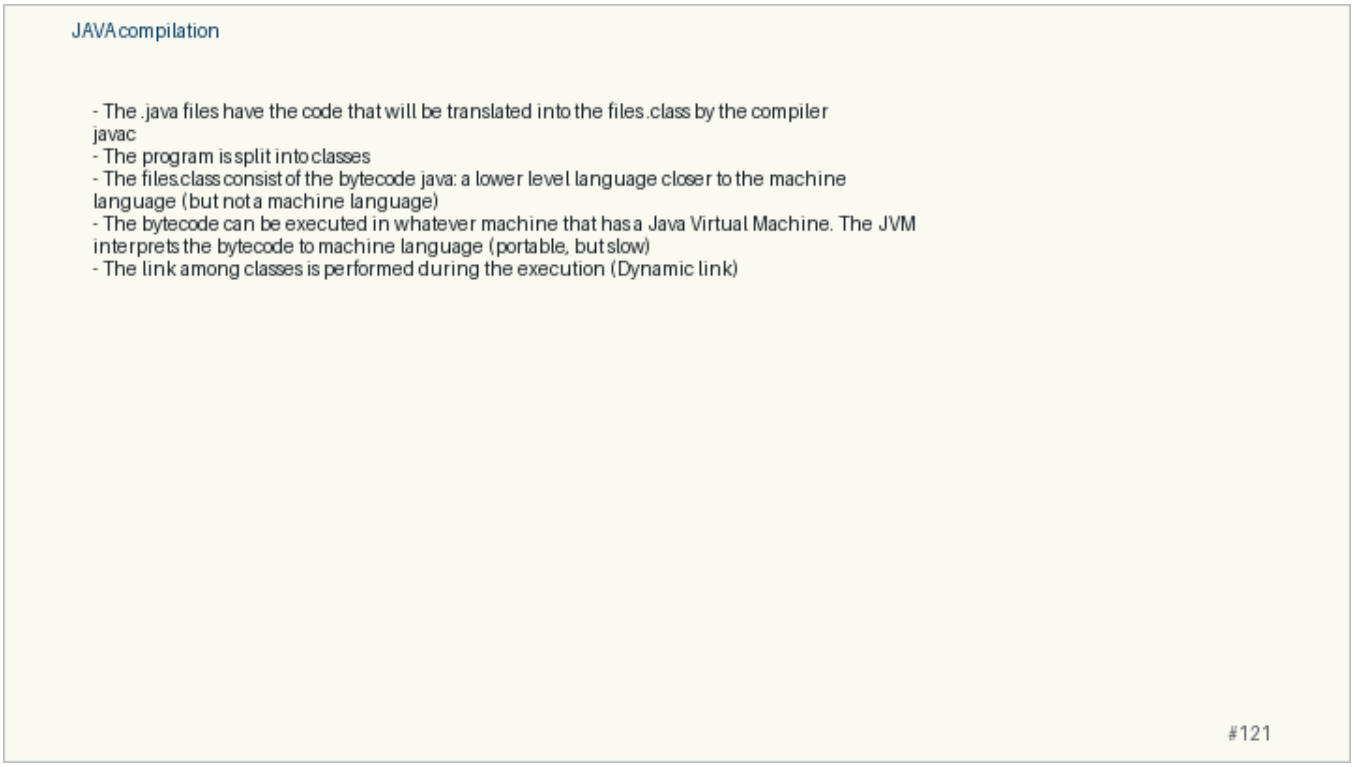
Question. What happens before an executable exists?

Core claim. Compilation is a pipeline; `make` records the dependency reasoning humans otherwise repeat by hand.

Target capability. Students can identify preprocess, compile, link, warning, and rebuild steps.

3.1 Main Path

- Start from the question: What happens before an executable exists?
- Build the model: Compilation is a pipeline; `make` records the dependency reasoning humans otherwise repeat by hand.
- End with a checkable action: Students can identify preprocess, compile, link, warning, and rebuild steps.
- Keep only this slide on the horizontal path during the live lecture.



3.2 Worked Example

- Use this as the live trace: Show the same program built manually, then with a Makefile, then after editing one header.
- Representative source slide: 126
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

3.3 Anecdote Or Motivation

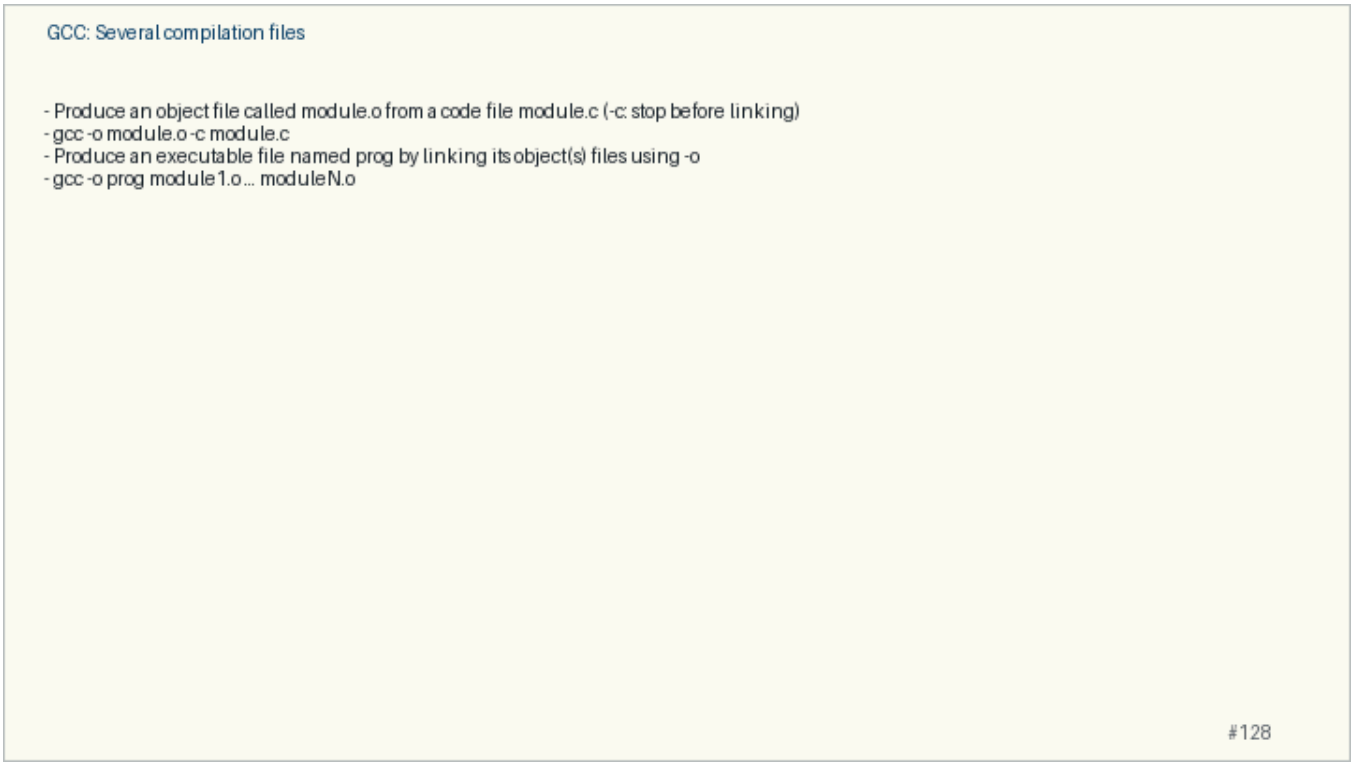
A good Makefile is a memory aid for the team; it documents what must be rebuilt and why.

- Use this only if students need a reason to care.

- If the room is already following, skip down to the check question.

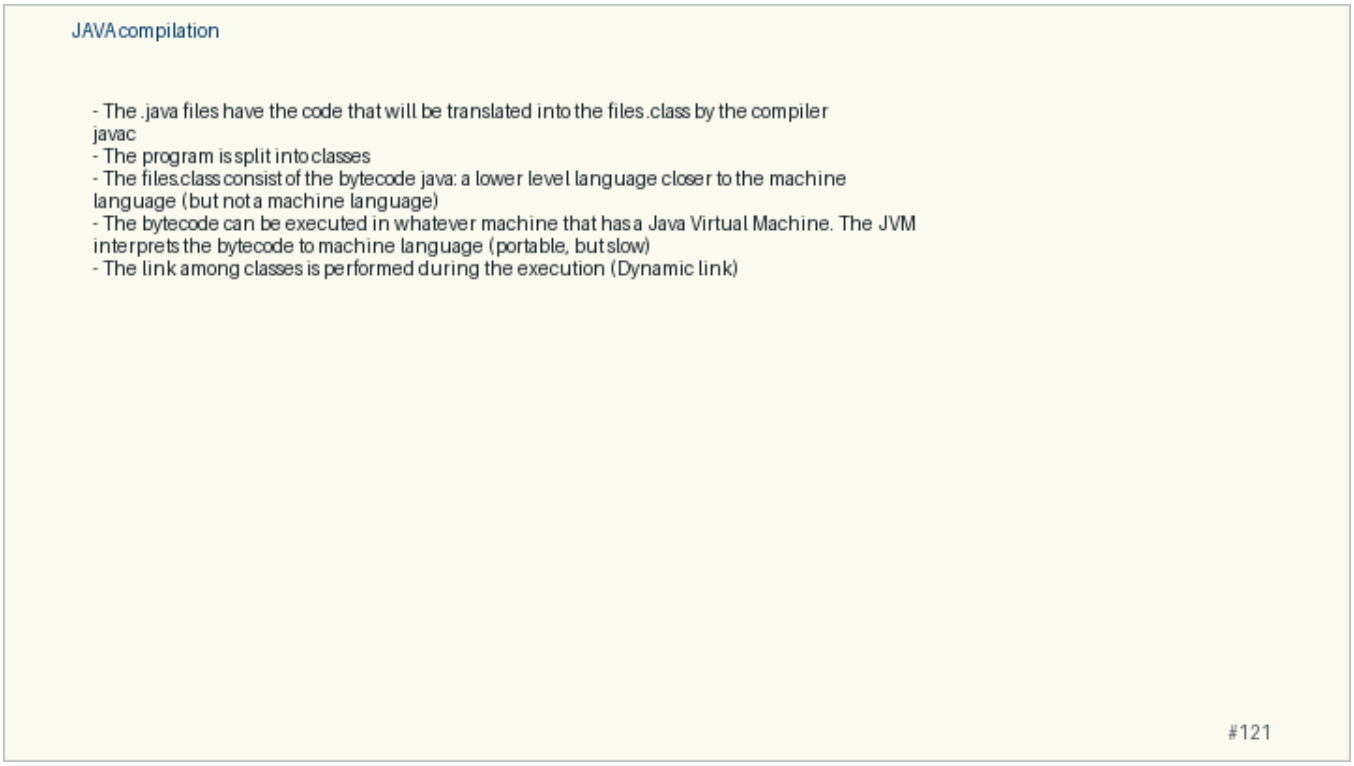
3.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.



3.5 Backup Index

Original slides 121-128



slide 121

JAVA compilation

- source Java
- Libraries (.jar)
- Java compiler
- File .class
- (bytecode)
- JAVA Virtual
- Machine
- File .class
- (bytecode)
- Execution
- File .class
- (bytecode)

#122

slide 122

C compilation

The call of the C compiler is associated with 3 tools
1st step: The pre-processor (CPP):
Handles the directives (#include, #define etc.)
2nd step: C compiler and assembler (GCC):
C compiler performs one-to-one translation of a compilation file (.c and .h) into an assembly file (.s).
This step is hidden by default (use gcc -S to see it)
Then, the assembler generates an object file (.o) with
the machine instructions of the functions defined in this.c file
information on the functions used in this.c file, but defined in another file

#123

slide 123

C compilation

- The generated binary depends on
- Architecture/processor
- AND
- operating system
- --> It is not portable !
- Possible to use "cross compilation"
- Use a machine to create an binary for another machine
- --march
- native
- i386

#124

slide 124

C compilation

- User C source file (.c)
- + headers (.h)

- Preprocessor +
- C compiler

- Assembly source

- Assembler

- File objet1 (.o)

- File objet2 (.o)

- Static Library file (.a)

- Dynamic Library
- (ex: .dll, .so)

- Loader (OS)

- Executable file

- Linker

- Program in machine
- language stored in RAM

- Execution

#125

slide 125

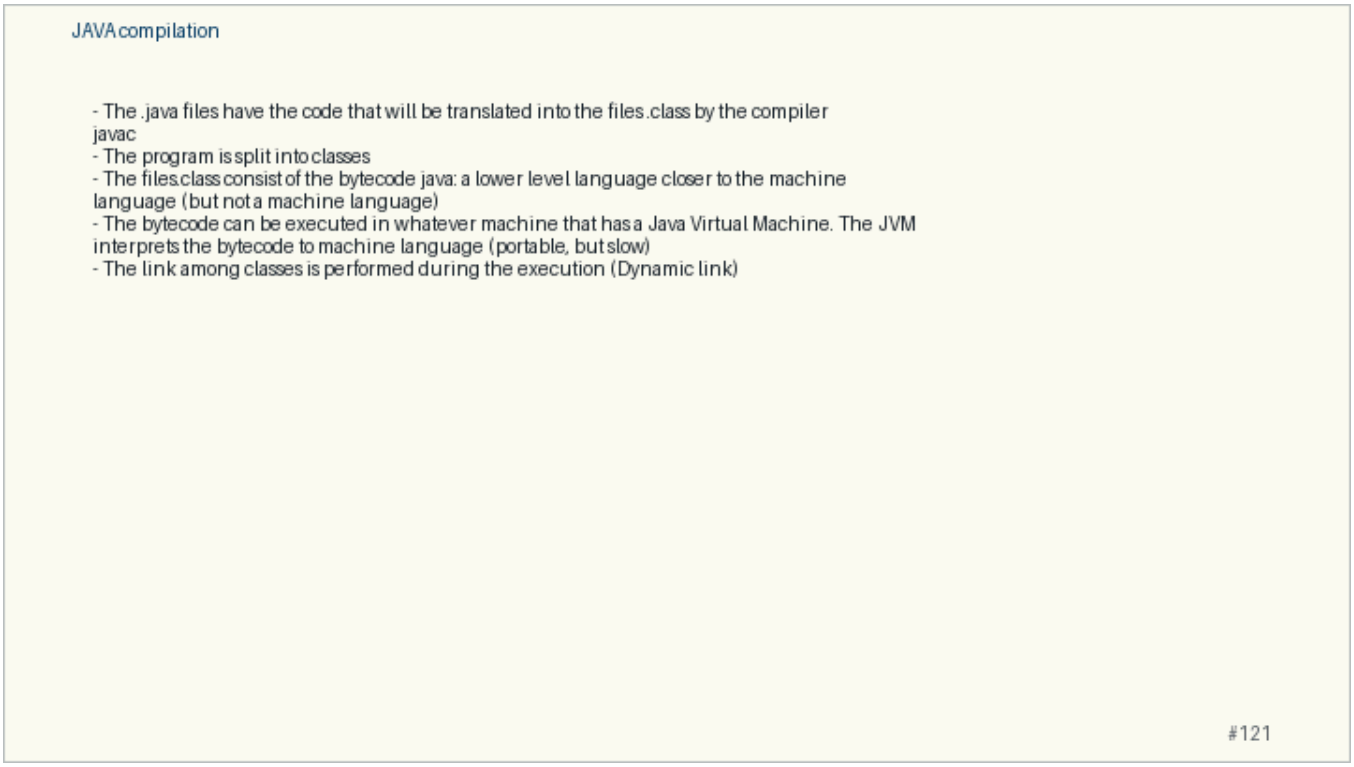
slide 126

Anchors: JAVA compilation; C compilation; C compilation: GCC; GCC: One compilation file; GCC: Several compilation files

3.6 Backup: JAVA compilation

Original slide 121

- The .java files have the code that will be translated into the files .class by the compiler javac
- The program is split into classes
- The files.class consist of the bytecode java: a lower level language closer to the machine language (but not a machine language)
- The bytecode can be executed in whatever machine that has a Java Virtual Machine. The JVM interprets the bytecode to machine language (portable, but slow)
- The link among classes is performed during the execution (Dynamic link)



3.7 Backup: JAVA compilation

Original slide 122

- source Java
- Libraries (.jar)
- Java compiler
- File .class
- (bytecode)
- JAVA Virtual
- Machine
- File .class
- (bytecode)
- Execution
- File .class
- (bytecode)

3.8 Backup: C compilation

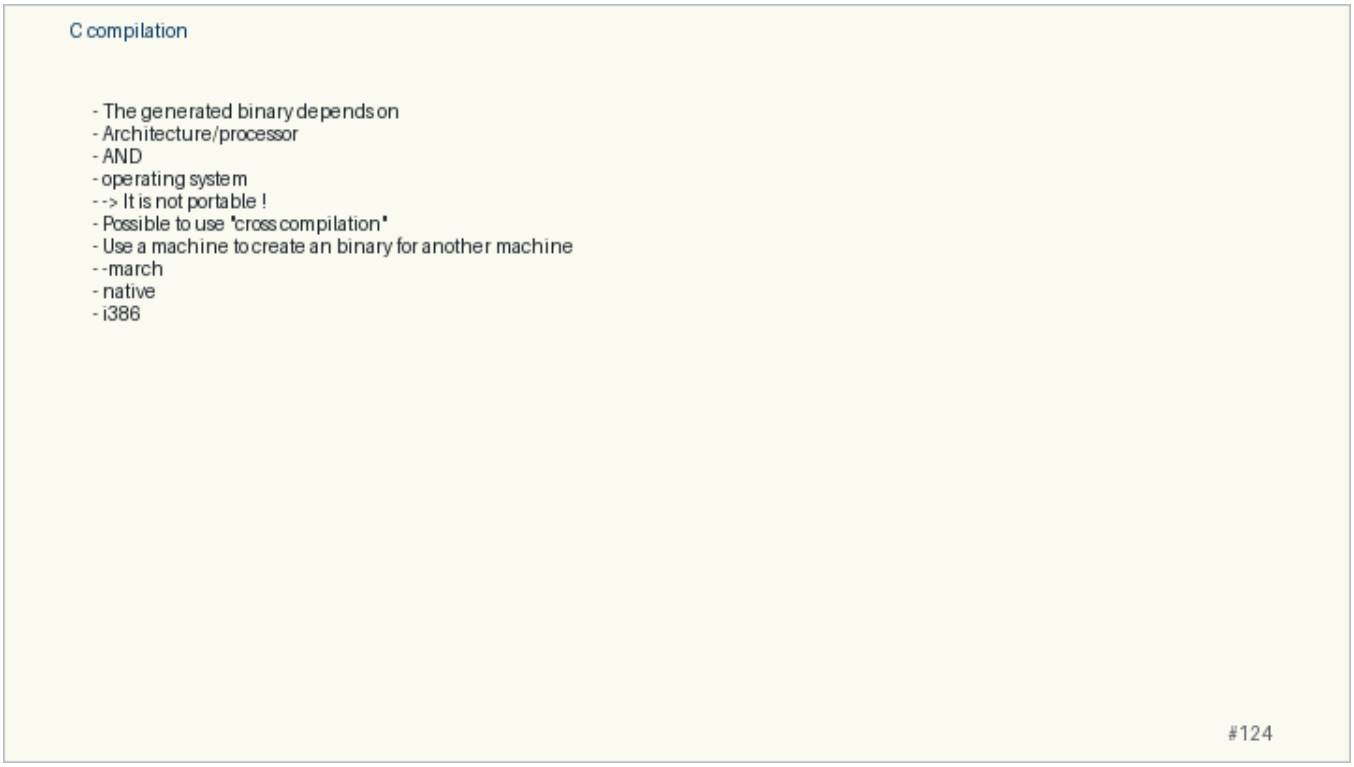
Original slide 123

```
The call of the C compiler is associated with 3 tools
1st step: The pre-processor (CPP):
Handles the directives (#include, #define etc.)
2nd step: C compiler and assembler (GCC):
C compiler performs one-to-one translation of a compilation file (.c and .h) into an assembly
file (.s).
This step is hidden by default (use gcc -S to see it)
Then, the assembler generates an object file (.o) with
the machine instructions of the functions defined in this .c file
information on the functions used in this .c file, but defined in another file
3rd step: The linker (LD/GCC):
Makes the link (resolves symbols) among the different program's objects (static linking) to
one executable binary
```

3.9 Backup: C compilation

Original slide 124

- The generated binary depends on
- Architecture/processor
- AND
- operating system
- -> It is not portable !
- Possible to use "cross compilation"
- Use a machine to create an binary for another machine
- -march
- native
- i386



3.10 Backup: C compilation

Original slide 125

- User C source file (.c)
- ▸ headers (.h)
- Dynamic Library
- (ex : .dll, .so)
- Preprocessor +
- C compiler
- Program in machine
- language stored in RAM
- Loader (OS)
- Assembly source
- Assembler
- Executable file
- File objet1 (.o)
- Linker
- File objet2 (.o)
- Execution
- Static Library file (.a)

3.11 Backup: C compilation: GCC

Original slide 126

- GNU (GNU is Not Unix!) Compiler Collection is free software:
- Design to target several machines
- Exist by default in Linux
- Syntax (see man gcc):
- gcc [-c] [-g] [-I dir] [-o nom] f1 ... fn
- Options
- -c : Produce an object file .o from .c file by stopping before linking
- -I dir : adds dir in the include path (used header files .h)
- -o nom : Specifies the name of output file. By default:
- object files have the same name as their code files
- Executable file: a.out
- -g : Inserts the information needed by the debugger

C compilation: GCC

- GNU (GNU is Not Unix!) Compiler Collection is free software:
- Design to target several machines
- Exist by default in Linux
- Syntax (see man gcc):
- gcc [-c] [-g] [-I dir] [-o nom] f1 ... fn
- Options
- -c : Produce an object file .o from .c file by stopping before linking
- -I dir : adds dir in the include path (used header files.h)
- -o nom : Specifies the name of output file. By default:
- object files have the same name as their code files

#126

3.12 Backup: GCC: One compilation file

Original slide 127

- Produce an executable called module from the file module.c
- gcc module.c -o module

GCC: One compilation file

- Produce an executable called module from the file module.c
- gcc module.c -o module

```
/*****************************
 * main.c
 *
 * le classique Hello World
 *****************************/

#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    printf("Hello World\n");
    return EXIT_SUCCESS;
}
```

```
bash-3.2$ ls
main.c
bash-3.2$ gcc main.c -o hello
bash-3.2$ ls
hello   main.c
bash-3.2$ ./hello
Hello World
bash-3.2$
```

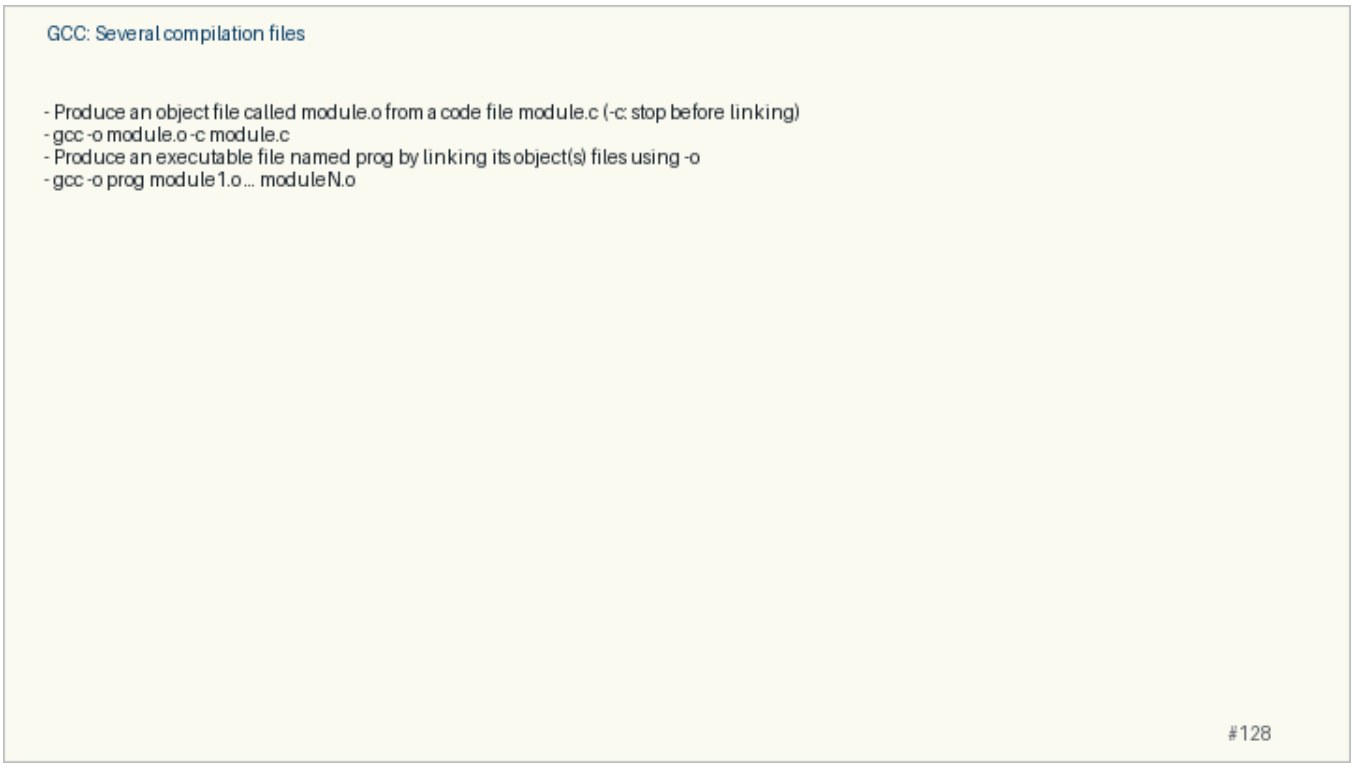
#127

3.13 Backup: GCC: Several compilation files

Original slide 128

- Produce an object file called module.o from a code file module.c (-c: stop before linking)
- gcc -o module.o -c module.c
- Produce an executable file named prog by linking its object(s) files using -o

- gcc -o prog module1.o ... moduleN.o



4 Make

Live pacing: about 8 min

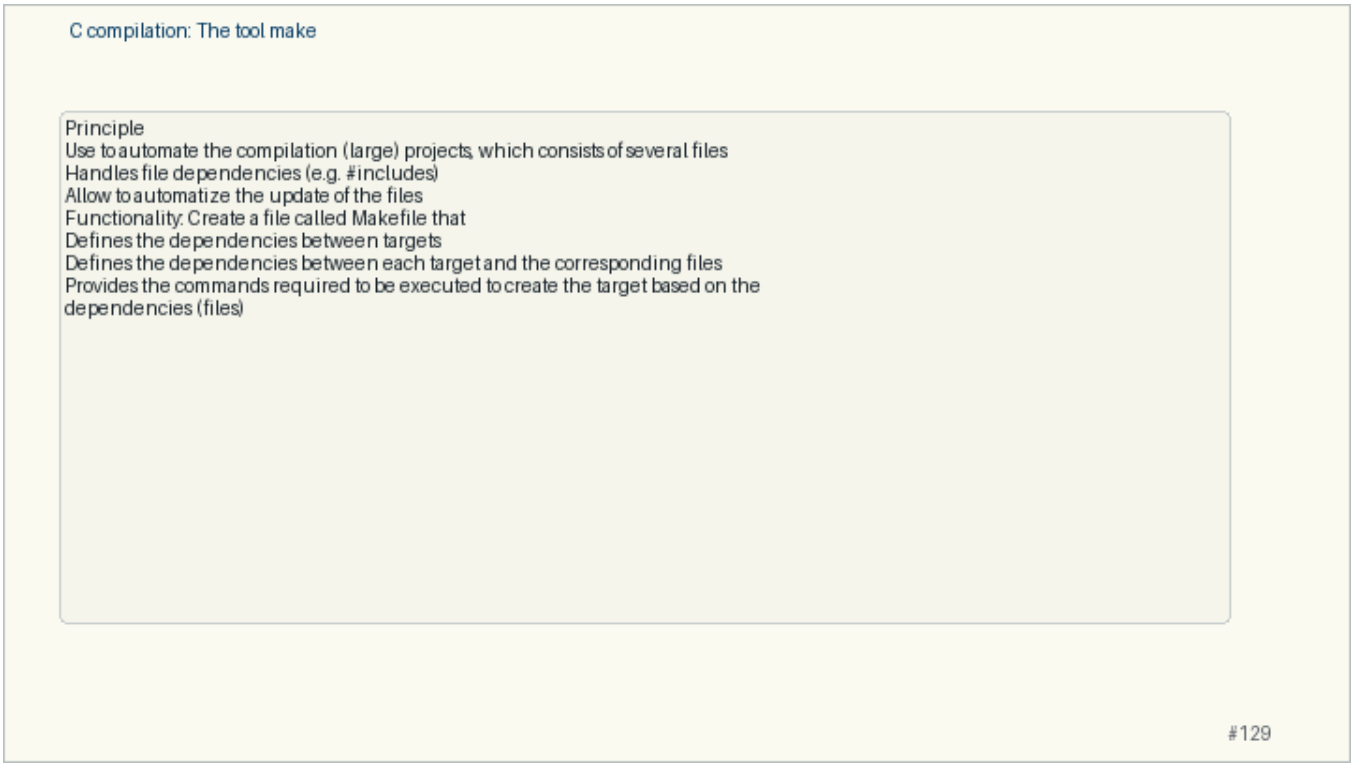
Question. What happens before an executable exists?

Core claim. Compilation is a pipeline; `make` records the dependency reasoning humans otherwise repeat by hand.

Target capability. Students can identify preprocess, compile, link, warning, and rebuild steps.

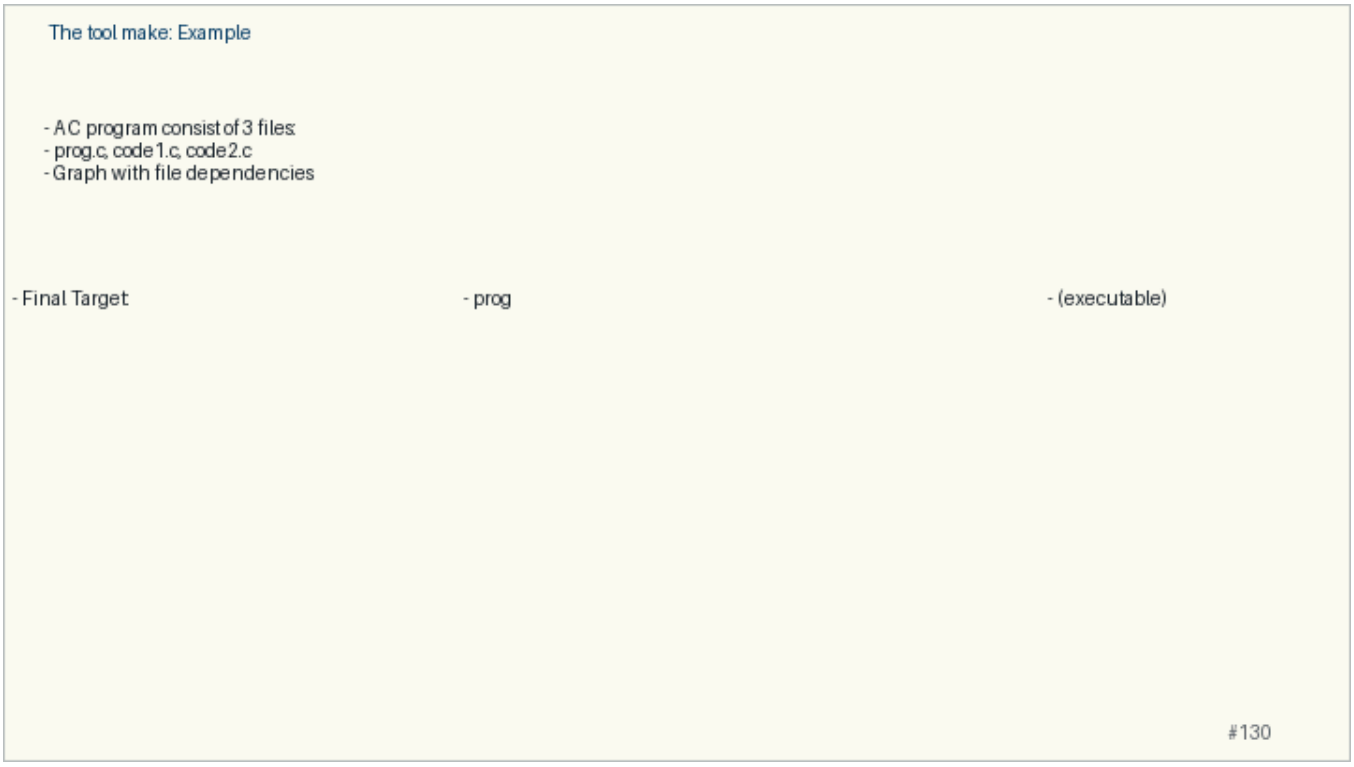
4.1 Main Path

- Start from the question: What happens before an executable exists?
- Build the model: Compilation is a pipeline; `make` records the dependency reasoning humans otherwise repeat by hand.
- End with a checkable action: Students can identify preprocess, compile, link, warning, and rebuild steps.
- Keep only this slide on the horizontal path during the live lecture.



4.2 Worked Example

- Use this as the live trace: Show the same program built manually, then with a Makefile, then after editing one header.
- Representative source slide: 130
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.



4.3 Anecdote Or Motivation

A good Makefile is a memory aid for the team; it documents what must be rebuilt and why.

- Use this only if students need a reason to care.

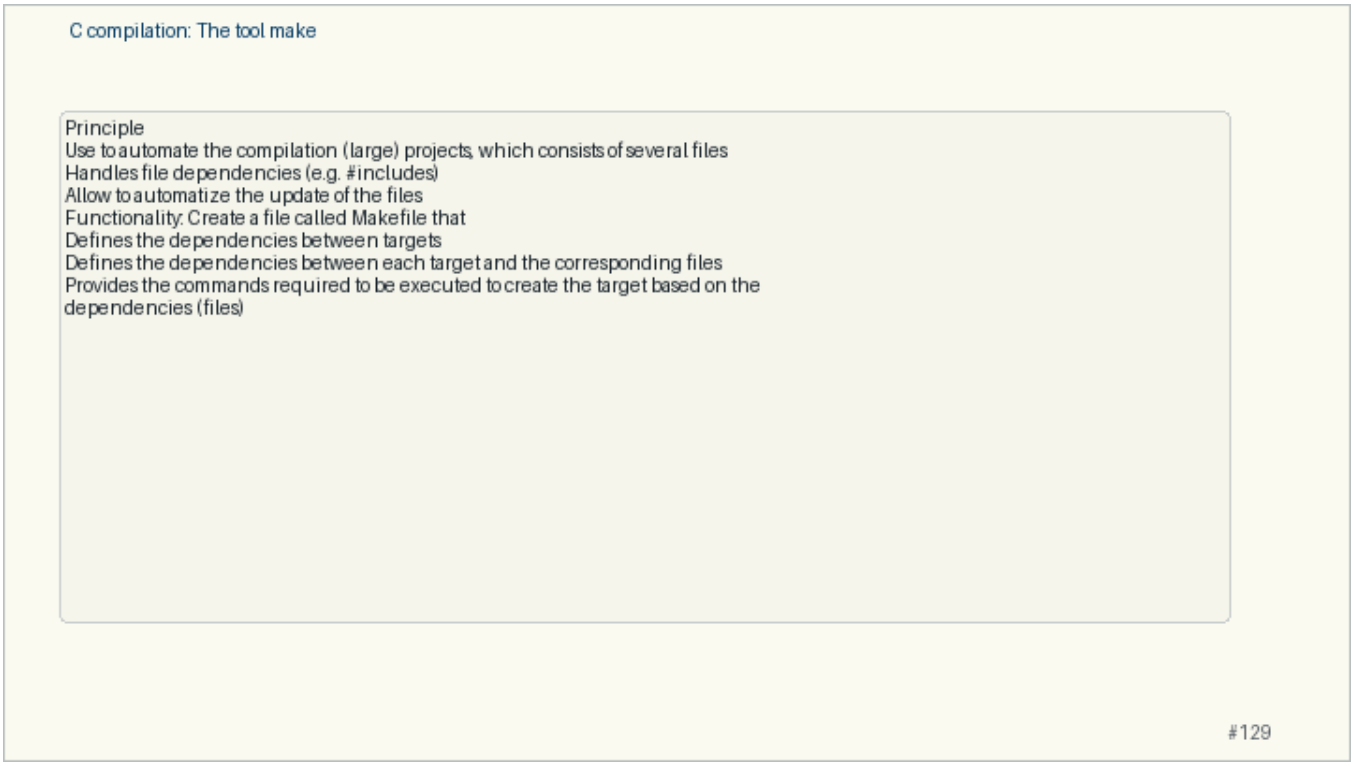
- If the room is already following, skip down to the check question.

4.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

4.5 Backup Index

Original slides 129-140



slide 129

The tool make: Example

- A C program consist of 3 files:
- prog.c, code1.c, code2.c
- Graph with file dependencies

- Final Target: - prog - (executable)

slide 130

The tool make: Example

- A C program consist of 3 files:
- prog.c, code1.c, code2.c
- Graph with file dependencies

- Final Target: - prog - (executable)

- prog.o - code1.o - code2.o - (object)

- To produce the executable file prog, the following command has to be executed :
- gcc -o prog prog.o code1.o code2.o
- The file prog depends on the object files prog.o, code1.o and code2.o

slide 131

The tool make: Example

- A C program consist of 3 files:
- prog.c, code1.c, code2.c
- Graph with file dependencies

- Final Target: - prog - (executable)

- prog.o - code1.o - code2.o - (object)

- prog.c - (source)

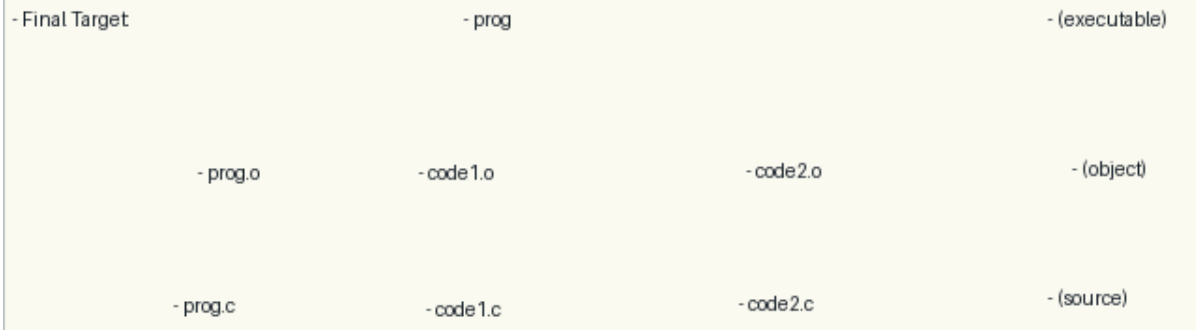
- To produce the object file prog.o, we have to execute
- gcc -c prog.c -o prog.o
- So, the file prog.o depends on the source file prog.c

#132

slide 132

The tool make: Example

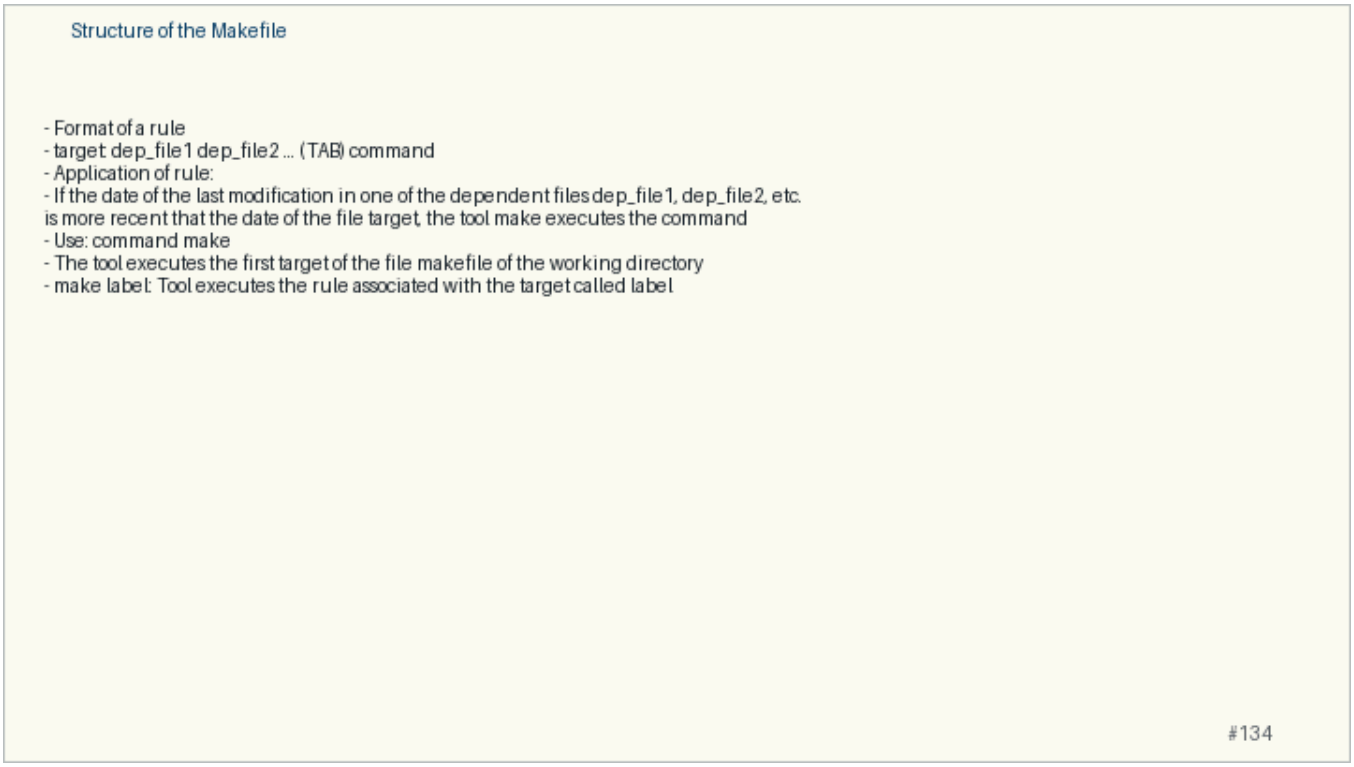
- A C program consist of 3 files:
- prog.c, code1.c, code2.c
- Graph with file dependencies



- The graph of file dependencies provides the definition of the targets, dependencies and commands of the makefile

#133

slide 133



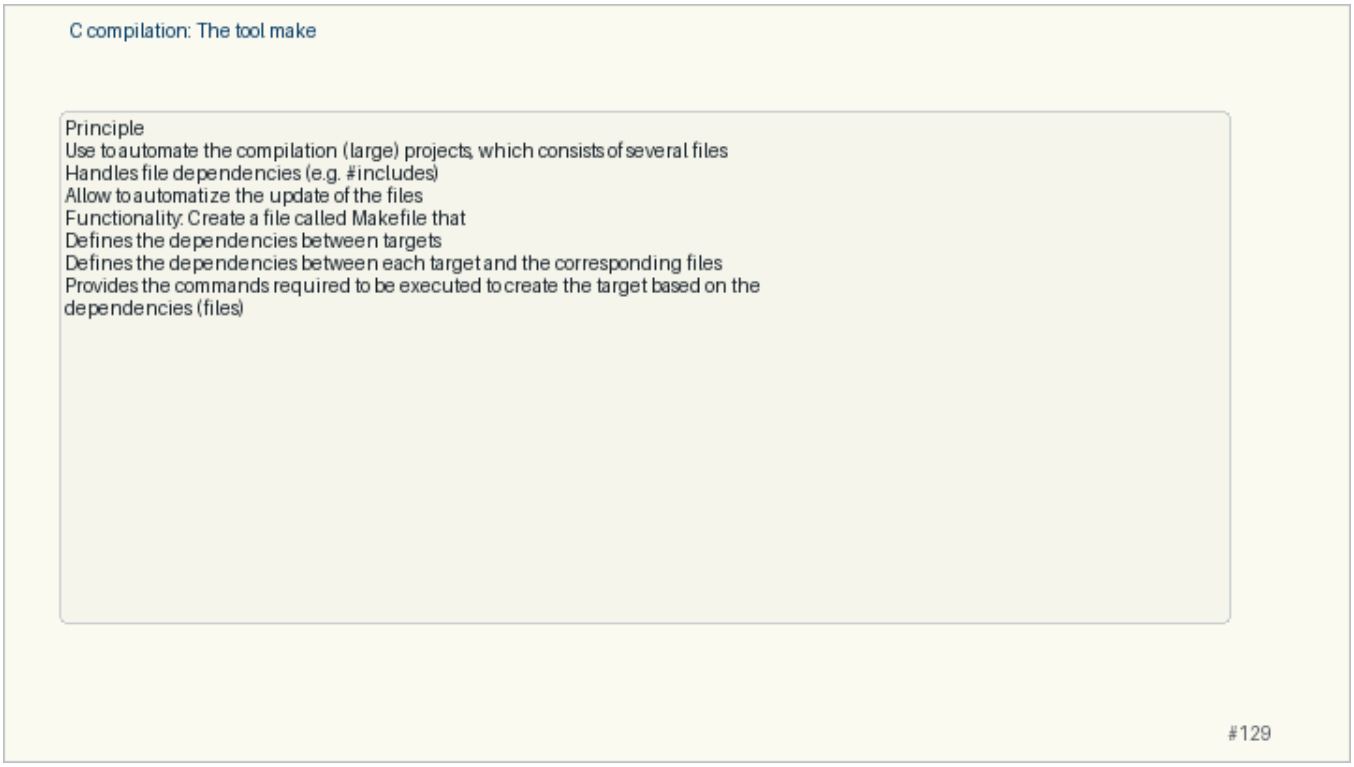
slide 134

Anchors: C compilation: The tool make; The tool make: Example; Structure of the Makefile; Structure of the Makefile: Example; Structure of the Makefile: Example 2

4.6 Backup: C compilation: The tool make

Original slide 129

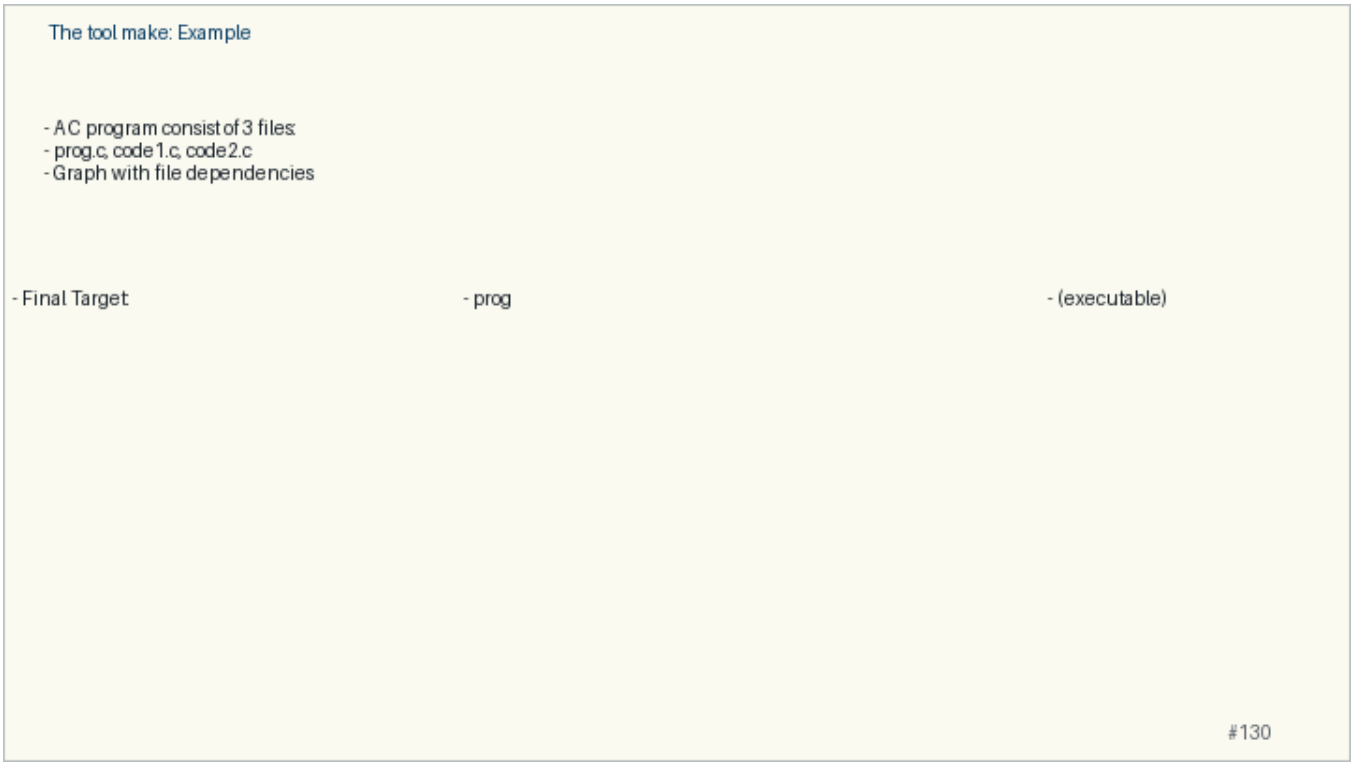
```
Principle
Use to automate the compilation (large) projects, which consists of several files
Handles file dependencies (e.g. #includes)
Allow to automatize the update of the files
Functionality: Create a file called Makefile that
Defines the dependencies between targets
Defines the dependencies between each target and the corresponding files
Provides the commands required to be executed to create the target based on the dependencies
(files)
```



4.7 Backup: The tool make: Example

Original slide 130

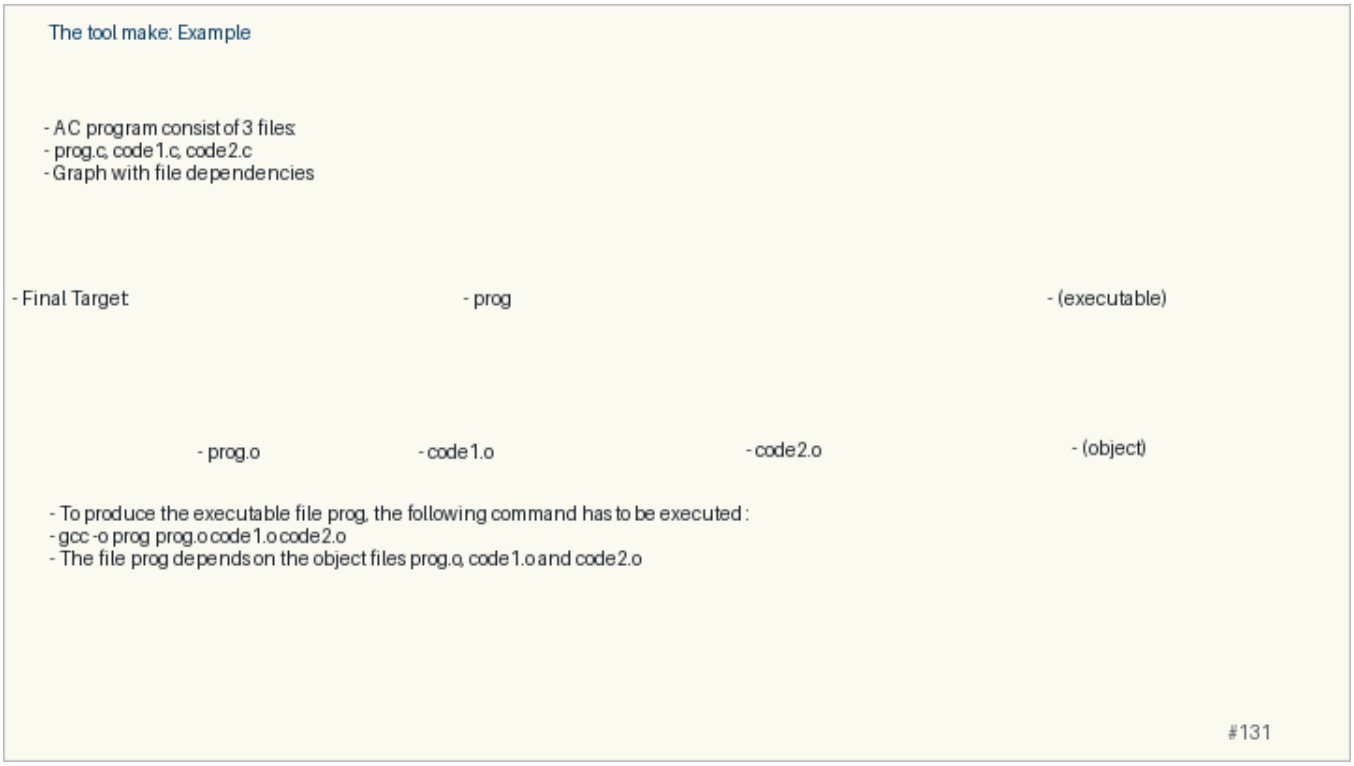
- A C program consist of 3 files:
- prog.c, code1.c, code2.c
- Graph with file dependencies
- prog
- Final Target:
- (executable)



4.8 Backup: The tool make: Example

Original slide 131

- A C program consist of 3 files:
- prog.c, code1.c, code2.c
- Graph with file dependencies
- prog
- Final Target:
- (executable)
- (object)
- code2.o
- code1.o
- prog.o
- To produce the executable file prog, the following command has to be executed :
- gcc -o prog prog.o code1.o code2.o
- The file prog depends on the object files prog.o, code1.o and code2.o

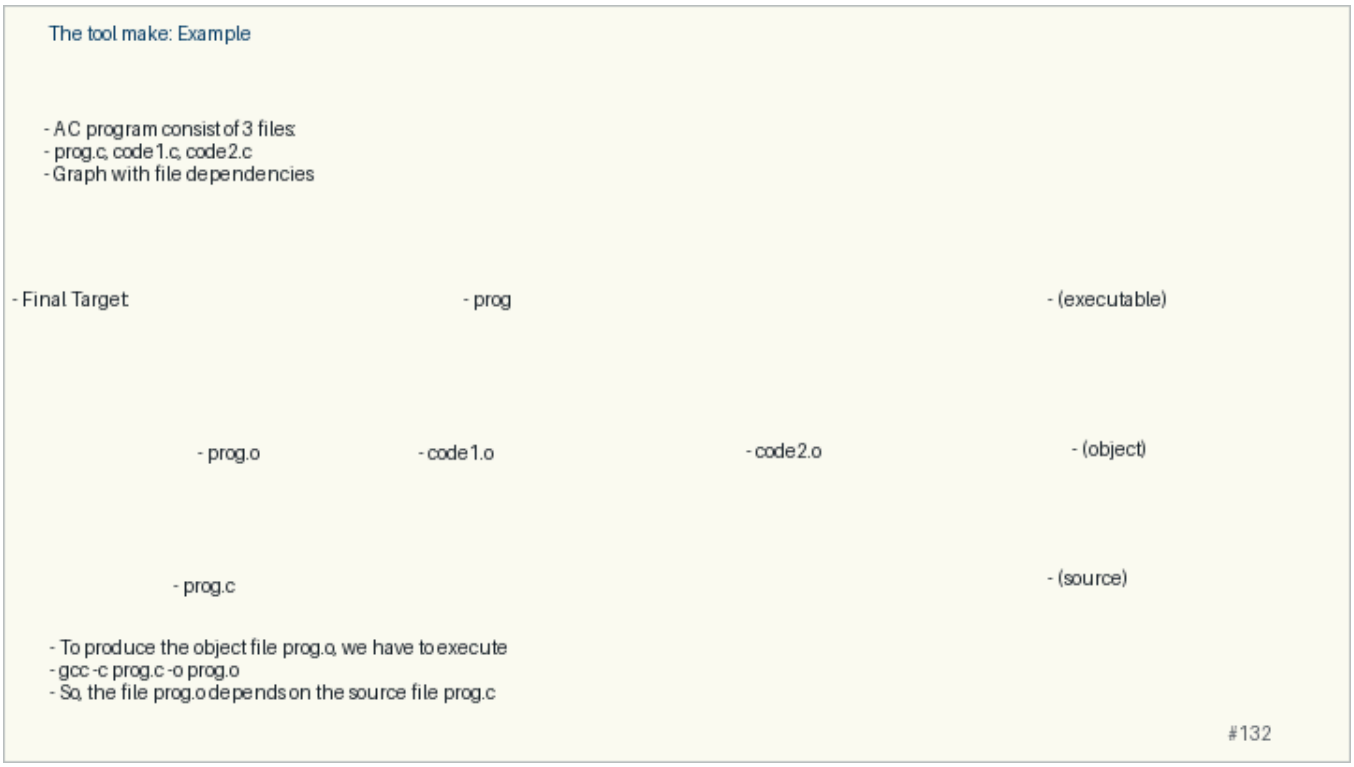


4.9 Backup: The tool make: Example

Original slide 132

- A C program consist of 3 files:
- prog.c, code1.c, code2.c
- Graph with file dependencies
- prog
- Final Target:
- (executable)

- (object)
- code2.o
- code1.o
- prog.o
- (source)
- prog.c
- To produce the object file prog.o, we have to execute
- gcc -c prog.c -o prog.o
- So, the file prog.o depends on the source file prog.c

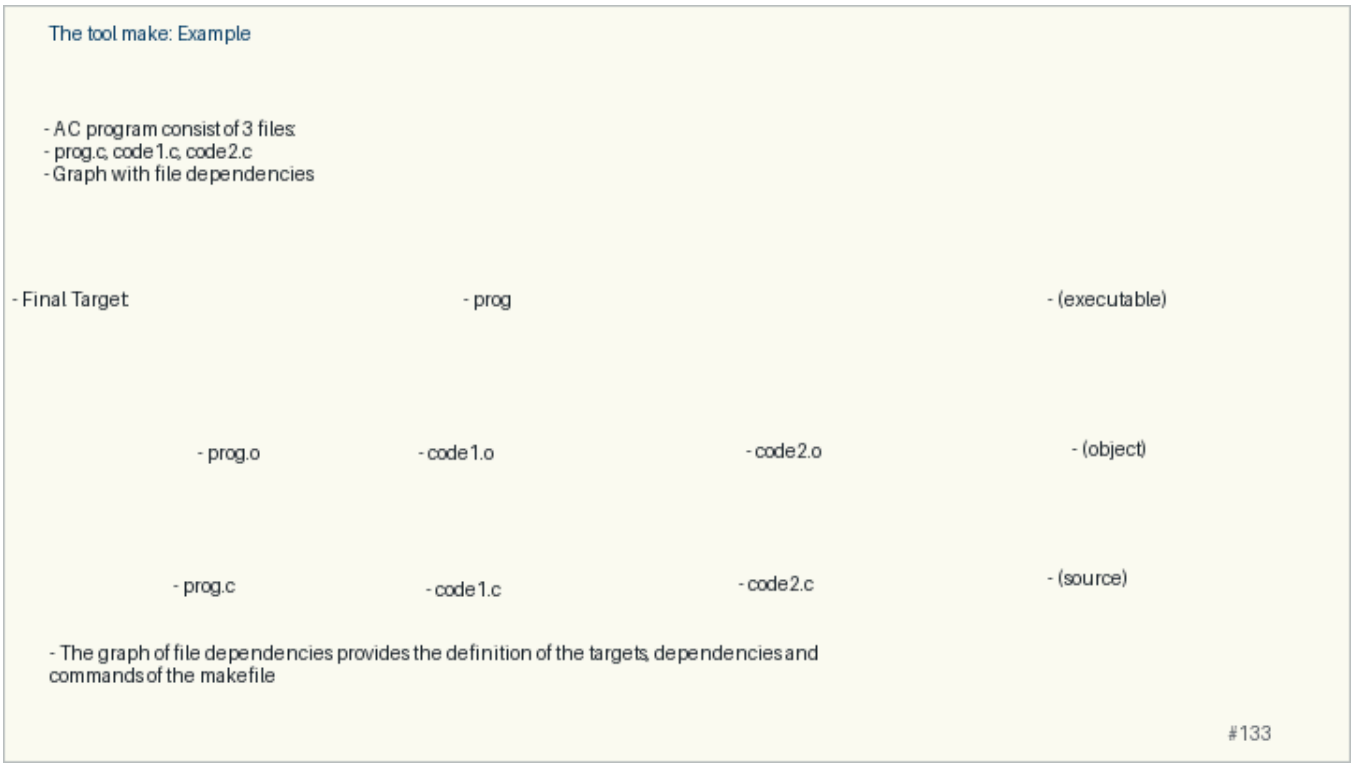


4.10 Backup: The tool make: Example

Original slide 133

- A C program consist of 3 files:
- prog.c, code1.c, code2.c
- Graph with file dependencies
- prog
- Final Target:
- (executable)
- (object)
- code2.o
- code1.o
- prog.o
- (source)

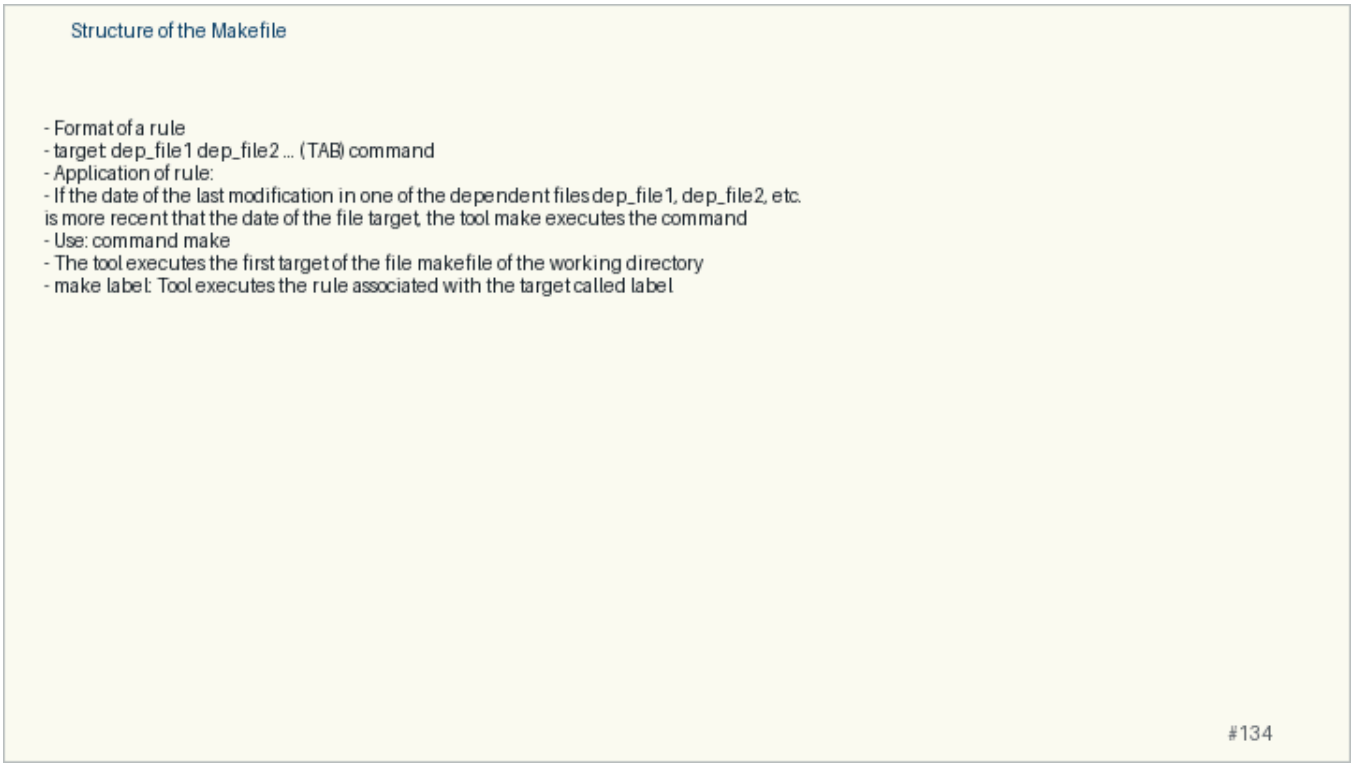
- code2.c
- prog.c
- code1.c
- The graph of file dependencies provides the definition of the targets, dependencies and commands of the makefile



4.11 Backup: Structure of the Makefile

Original slide 134

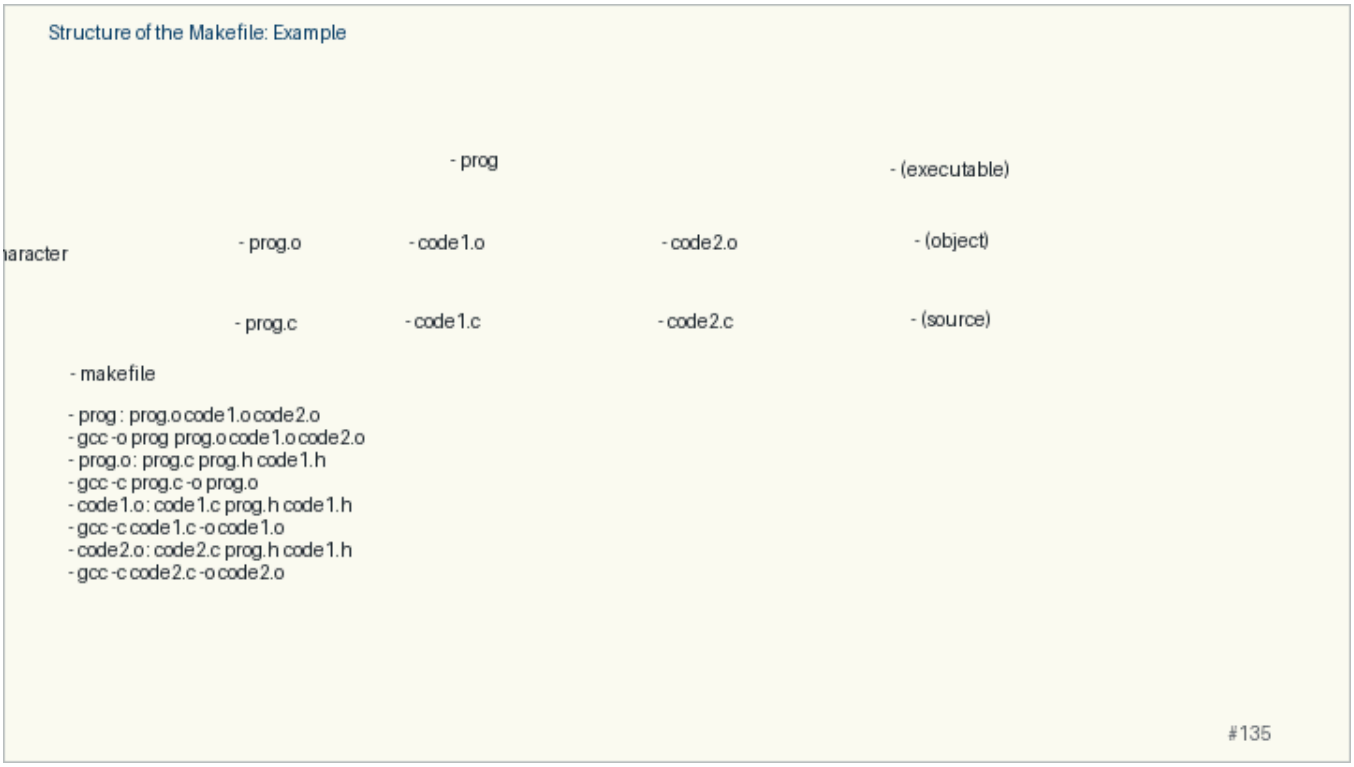
- Format of a rule
- target: dep_file1 dep_file2 ... (TAB) command
- Application of rule:
- If the date of the last modification in one of the dependent files dep_file1, dep_file2, etc. is more recent that the date of the file target, the tool make executes the command
- Use: command make
- The tool executes the first target of the file makefile of the working directory
- make label: Tool executes the rule associated with the target called label



4.12 Backup: Structure of the Makefile: Example

Original slide 135

- prog
- (executable)
- (object)
- code2.o
- code1.o
- prog.o
- TAB character
- (source)
- code2.c
- code1.c
- prog.c
- makefile
- prog : prog.o code1.o code2.o
- gcc -o prog prog.o code1.o code2.o
- prog.o : prog.c prog.h code1.h
- gcc -c prog.c -o prog.o
- code1.o : code1.c prog.h code1.h
- gcc -c code1.c -o code1.o
- code2.o : code2.c prog.h code1.h
- gcc -c code2.c -o code2.o

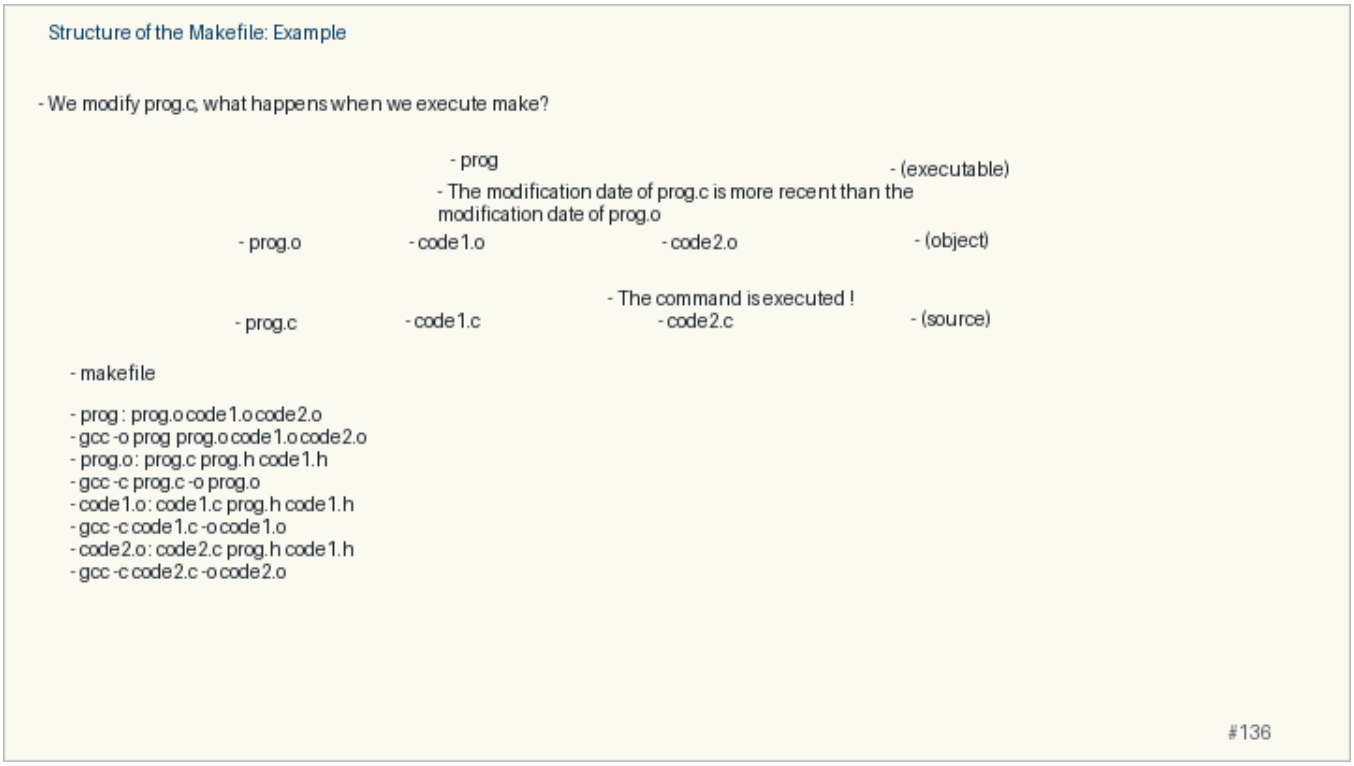


4.13 Backup: Structure of the Makefile: Example

Original slide 136

- We modify prog.c, what happens when we execute make?
- prog
- (executable)
- The modification date of prog.c is more recent than the modification date of prog.o
- (object)
- code2.o
- code1.o
- prog.o
- The command is executed !
- (source)
- code2.c
- code1.c
- prog.c
- makefile
- prog : prog.o code1.o code2.o
- gcc -o prog prog.o code1.o code2.o
- prog.o : prog.c prog.h code1.h
- gcc -c prog.c -o prog.o
- code1.o : code1.c prog.h code1.h
- gcc -c code1.c -o code1.o

- code2.o : code2.c prog.h code1.h
- gcc -c code2.c -o code2.o

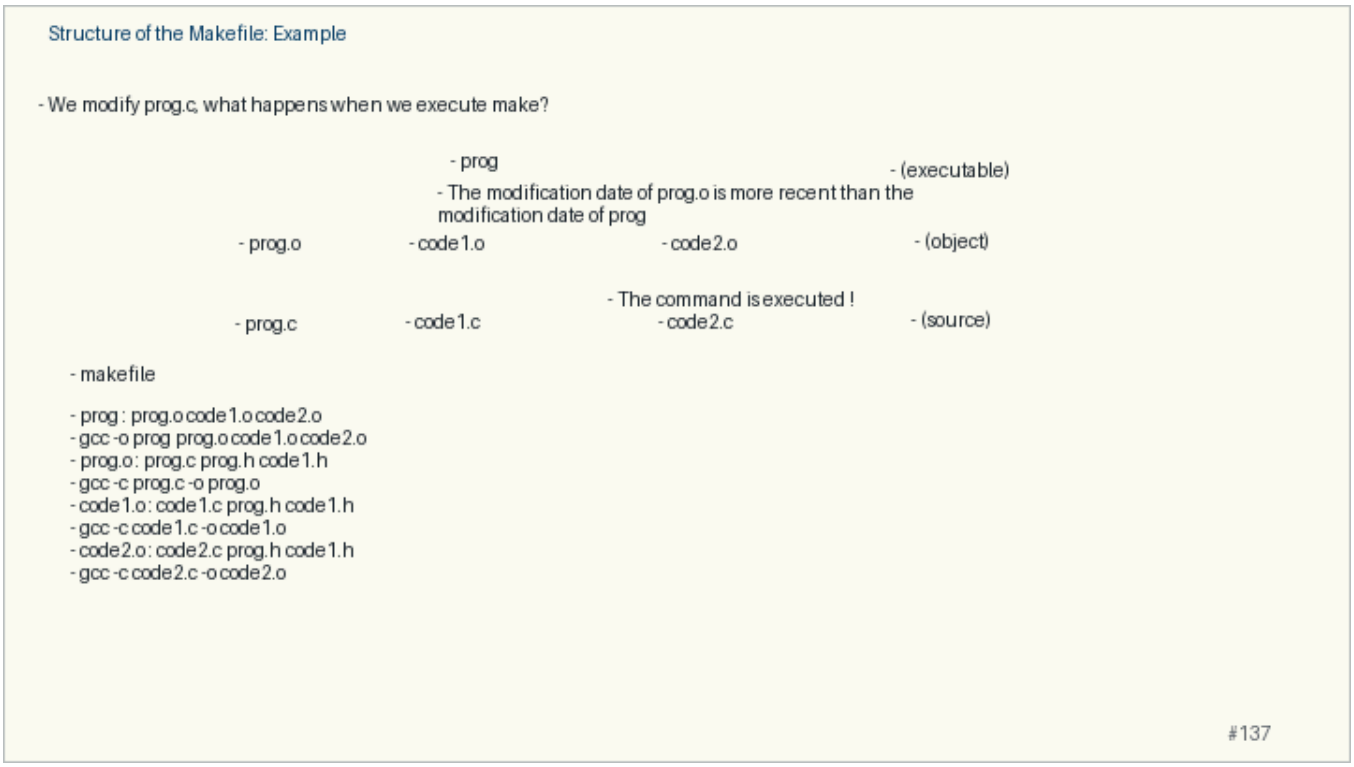


4.14 Backup: Structure of the Makefile: Example

Original slide 137

- We modify prog.c, what happens when we execute make?
- prog
- (executable)
- The modification date of prog.o is more recent than the modification date of prog
- (object)
- code2.o
- code1.o
- prog.o
- The command is executed !
- (source)
- code2.c
- code1.c
- prog.c
- makefile
- prog : prog.o code1.o code2.o
- gcc -o prog prog.o code1.o code2.o
- prog.o : prog.c prog.h code1.h
- gcc -c prog.c -o prog.o

- code1.o : code1.c prog.h code1.h
- gcc -c code1.c -o code1.o
- code2.o : code2.c prog.h code1.h
- gcc -c code2.c -o code2.o



4.15 Backup: Structure of the Makefile: Example

Original slide 138

- Add a clean target: Remove object files, executable
- Use: make clean
- makefile

```
prog : prog.o code1.o code2.o
gcc -o prog prog.o code1.o code2.o
prog.o : prog.c prog.h code1.h
gcc -c prog.c -o prog.o
code1.o : code1.c prog.h code1.h
gcc -c code1.c -o code1.o
code2.o : code2.c prog.h code1.h
gcc -c code2.c -o code2.o
clean :
rm *.o prog
```

4.16 Backup: Structure of the Makefile: Example 2

Original slide 139

```
hello: hello.o main.o
gcc -o hello hello.o main.o
hello.o: hello.c hello.h
gcc -c hello.c -o hello.o
main.o: main.c hello.h
gcc -c main.c -o main.o
clean:
rm *.o
```

4.17 Backup: Structure of the Makefile: Example 2

Original slide 140

```
CC=gcc
CFLAGS=-g -Wall -I.
DEP=hello.h
OBJ=hello.o main.o
EXEC=hello
all: $(EXEC)
$(EXEC): $(OBJ)
$(CC) -o $@ $^
%.o: %.c $(DEP)
$(CC) -c $(CFLAGS) $< -o $@
.PHONY: clean mrproper
clean:
rm *.o
mrproper: clean
rm $(EXEC)
```

5 Diagnostics and Tools

Live pacing: about 8 min

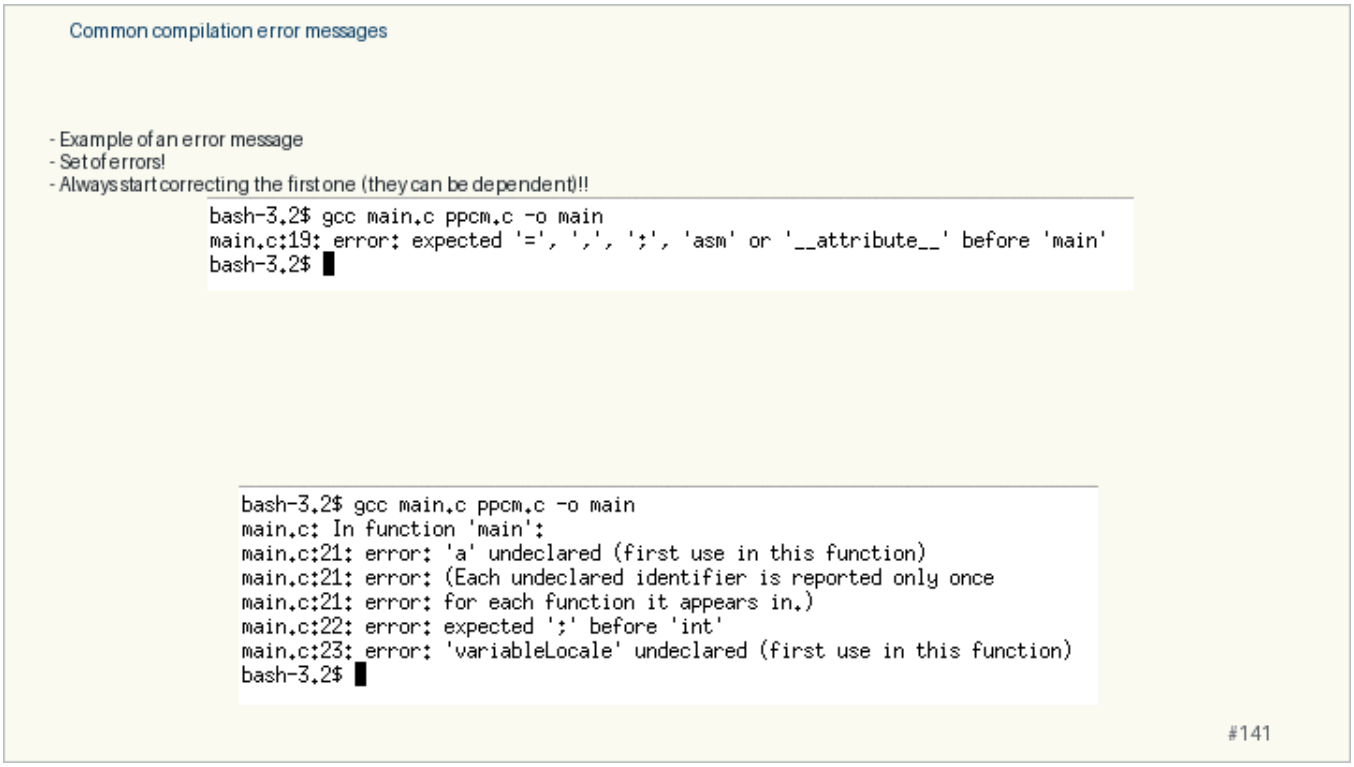
Question. What does the operating system do between the program and the machine?

Core claim. The OS gives a program controlled access to files, processes, memory, and devices.

Target capability. Students can place shell commands, processes, files, and permissions in the OS layer.

5.1 Main Path

- Start from the question: What does the operating system do between the program and the machine?
- Build the model: The OS gives a program controlled access to files, processes, memory, and devices.
- End with a checkable action: Students can place shell commands, processes, files, and permissions in the OS layer.
- Keep only this slide on the horizontal path during the live lecture.



5.2 Worked Example

- Use this as the live trace: Compare `ls`, an editor, and a compiled program: all are processes asking the OS for services.
- Representative source slide: 146
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

5.3 Anecdote Or Motivation

A useful mental image: user programs are guests; the OS is the host that owns the keys.

- Use this only if students need a reason to care.

- If the room is already following, skip down to the check question.

5.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Code style is very important!

- Understanding the code written by someone else is a hard job!
- Use a universal language (readable by everyone)
- Decide a code style and be CONSISTENT
- Comments
- Documentation: Programmers that want to use the code
- Implementation: Programmers that maintain the code, i.e. they have to understand your implementation
- Names that mean something
- int z Vs int counter

#147

5.5 Backup Index

Original slides 141-147

Common compilation error messages

- Example of an error message
- Set of errors!
- Always start correcting the first one (they can be dependent)!!

```
bash-3.2$ gcc main.c ppcm.c -o main
main.c:19: error: expected '=', ',', ';', 'asm' or '__attribute__' before 'main'
bash-3.2$
```

```
bash-3.2$ gcc main.c ppcm.c -o main
main.c: In function 'main':
main.c:21: error: 'a' undeclared (first use in this function)
main.c:21: error: (Each undeclared identifier is reported only once
main.c:21: error: for each function it appears in.)
main.c:22: error: expected ';' before 'int'
main.c:23: error: 'variableLocale' undeclared (first use in this function)
bash-3.2$
```

#141

slide 141

Debugging

Usual bugs
Infinite loops
Loop boundaries
Non-Initialized variables
Segmentation faults
Algorithmic faults
...
Tools to support the correction of the code
The debugger is a software that helps the developer in the analysis of the bugs of a program
Examples: GDB, valgrind ...

slide 142

Tool GDB

With the GDB, we can
Start the execution of the program
Stop the execution of the program based on some conditions
Examine what happened when the program stopped
Execute the program step-by-step
Perform modifications during the execution of the program
Use: Make the link between the program and the debugger
Add the information of debug during compilation
Option -g and -ggdb

slide 143

Tool GDB Commands

Start execution: run
Show the stack of the call: backtrace
Break Point : break (help breakpoints)
Continue execution : cont
Execute one instruction : step
Display the variables
Display nom_variable
watch nom_variable (allows to monitor)
info locals (local variables)
...
GDB Quick Reference Sheet !!!

#144

slide 144

Tool Valgrind

With Valgrind we can identify memory leaks
Example of a memory leak
Use: valgrind --tool=memcheck ./test

```c
/******************
 * test.c
 ******************/
#include <stdlib.h>
int main()
{
    int a=2;
    int* p = (int*) malloc(1024);
    p=&a;
    return 0;
}
```

#145

slide 145

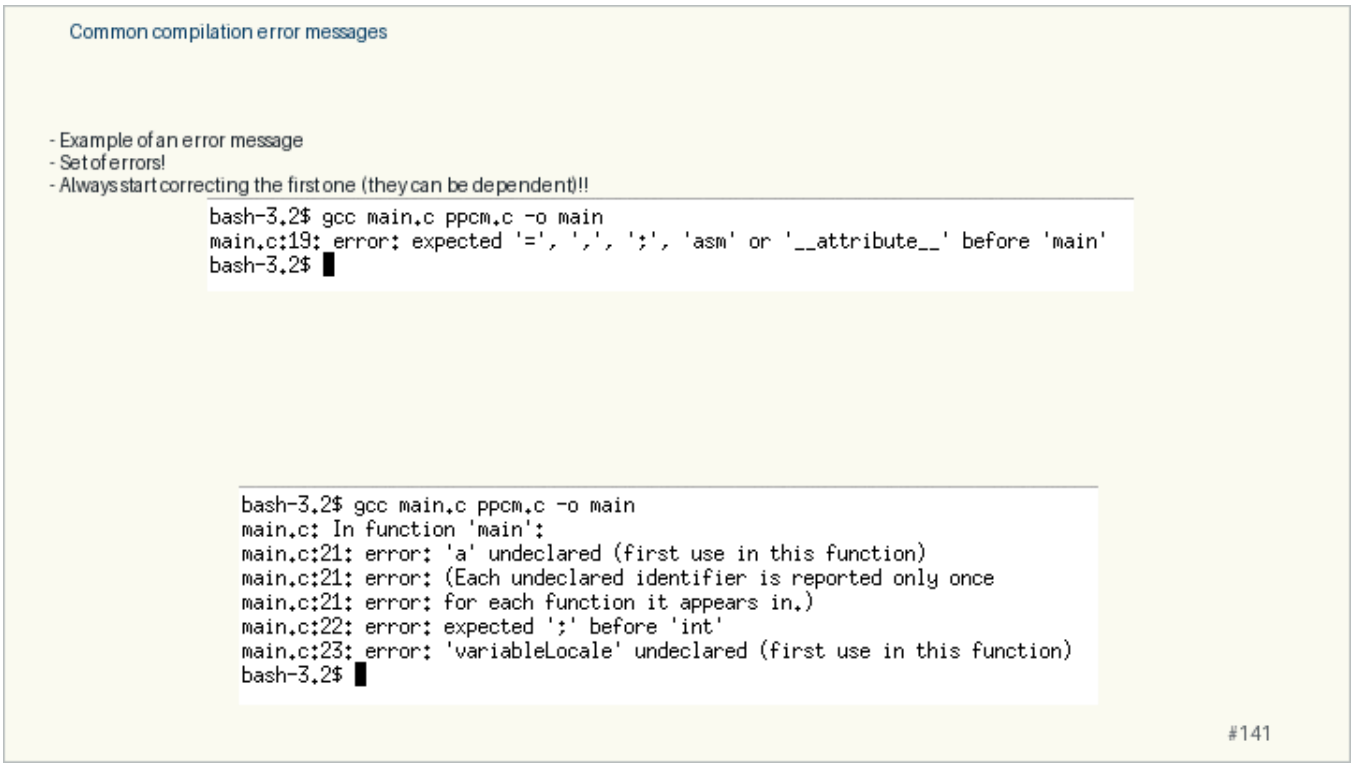
slide 146

Anchors: Common compilation error messages; Debugging; Tool GDB; Tool GDB: Commands; Tool Valgrind; ... before going in more details about the C syntax...

5.6 Backup: Common compilation error messages

Original slide 141

- Example of an error message
- Set of errors!
- Always start correcting the first one (they can be dependent)!!



5.7 Backup: Debugging

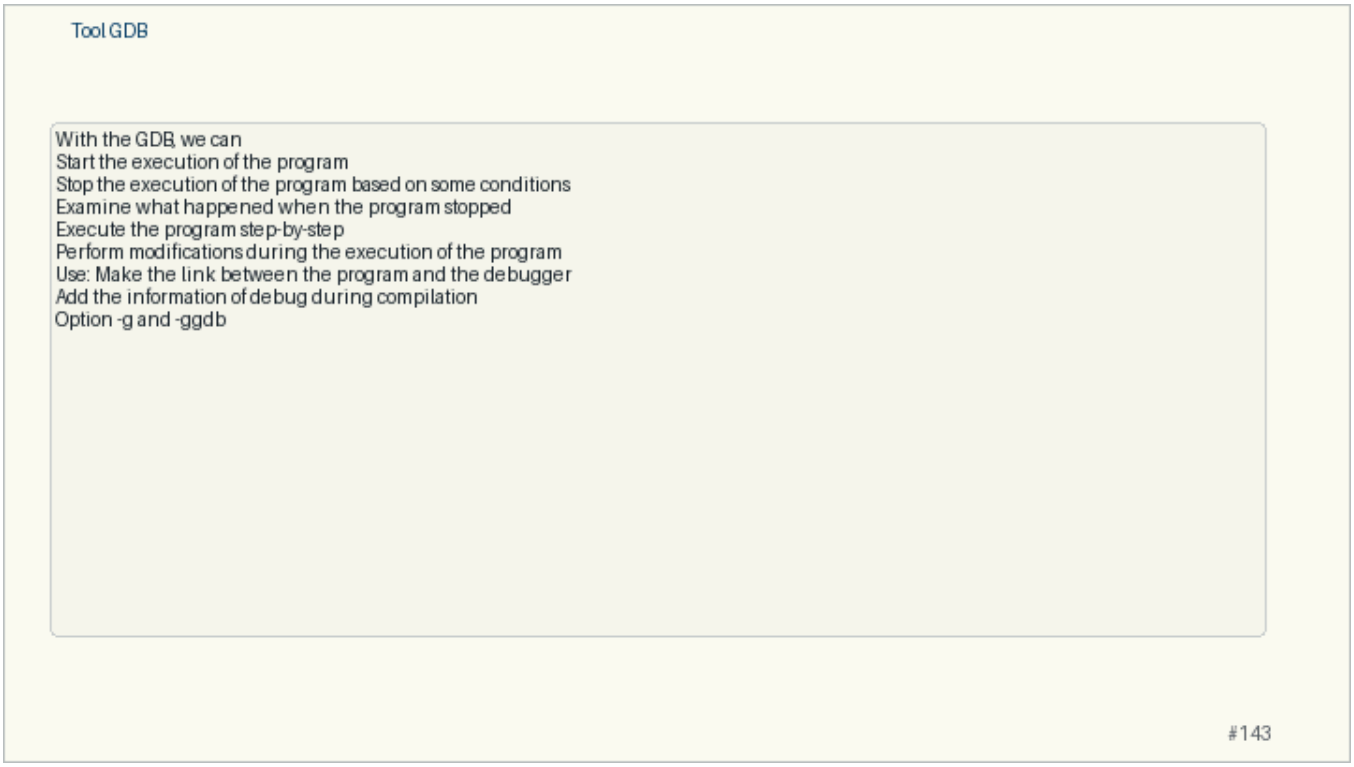
Original slide 142

```
Usual bugs
Infinite loops
Loop boundaries
Non-Initialized variables
Segmentation faults
Algorithmic faults
...
Tools to support the correction of the code
The debugger is a software that helps the developer in the analysis of the bugs of a program
Examples : GDB, valgrind ...
```

5.8 Backup: Tool GDB

Original slide 143

```
With the GDB, we can
Start the execution of the program
Stop the execution of the program based on some conditions
Examine what happened when the program stopped
Execute the program step-by-step
Perform modifications during the execution of the program
Use: Make the link between the program and the debugger
Add the information of debug during compilation
Option -g and -ggdb
```



5.9 Backup: Tool GDB: Commands

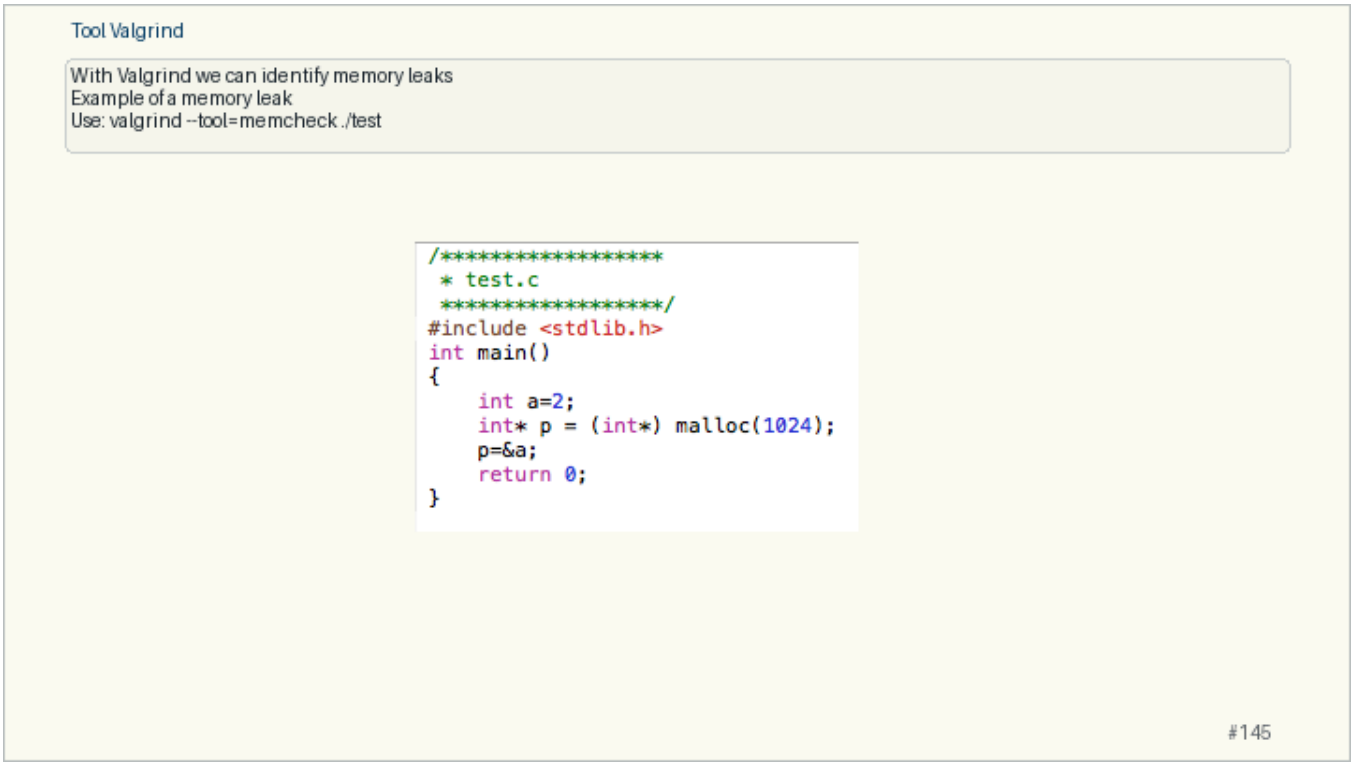
Original slide 144

```
Start execution: run
Show the stack of the call: backtrace
Break Point : break (help breakpoints)
Continue execution : cont
Execute one instruction : step
Display the variables
Display nom_variable
watch nom_variable (allows to monitor)
info locals (local variables)
...
GDB Quick Reference Sheet !!!
```

5.10 Backup: Tool Valgrind

Original slide 145

```
With Valgrind we can identify memory leaks
Example of a memory leak
Use: valgrind --tool=memcheck ./test
```



5.11 Backup: ...before going in more details about the C syntax...

Original slide 146

5.12 Backup: Code style is very important!

Original slide 147

- Understanding the code written by someone else is a hard job!
- Use a universal language (readable by everyone)
- Decide a code style and be CONSISTENT
- Comments
- Documentation: Programmers that want to use the code
- Implementation: Programmers that maintain the code, i.e. they have to understand your implementation
- Names that mean something
- int z Vs int counter